Comprehensive Guide to Running LLMs Locally

From Architecture to Deployment:
A Deep Technical Reference

May 2026

Contents

1 Introduction **11**

2 Efficient Base Model Architecture **13**
2.1 RMSNorm (Root Mean Square Layer Normalization) 13
2.1.1 Motivation 13
2.1.2 Mathematical Definition 13
2.1.3 Why It Works 13
2.1.4 Implementation 14
2.2 SwiGLU Activation 14
2.2.1 Background: Gated Linear Units 14
2.2.2 SwiGLU Definition 14
2.2.3 Why SwiGLU Outperforms ReLU/GELU 14
2.2.4 Variants of Gated Activations 15
2.2.5 Weight Scaling 15
2.3 RoPE: Rotary Positional Embeddings 15
2.3.1 Problem With Absolute Positional Encodings 15
2.3.2 Mathematical Foundation: Rotation in 2D 15
2.3.3 Efficient Implementation 16
2.3.4 RoPE Variants for Context Length Extension 16
2.3.5 PyTorch Implementation (Complex Number Approach) 17
2.4 Grouped Query Attention (GQA) 17
2.4.1 The Attention Hierarchy: MHA → MQA → GQA 17
2.4.2 Formal Definition 17
2.4.3 KV-Cache Memory Comparison 18
2.4.4 Training Considerations 18
2.4.5 Relationship to Other KV-Cache Reduction Methods 18
2.5 Tied Embeddings and Shared Layers 18
2.5.1 Input-Output Embedding Tying (Weight Tying) 18
2.5.2 Cross-Layer Parameter Sharing (ALBERT-style) 19
2.6 Other Architecture Optimizations 19
2.6.1 Parallel Attention/FFN (PaLM-style) 19
2.6.2 Pre-Norm vs Post-Norm 19
2.6.3 Untied Pre-Norm Gains 19
2.6.4 Summary of Efficient Architecture Components 20

3 Attention Mechanisms: A Complete Taxonomy **21**
3.1 Standard Scaled Dot-Product Attention 21
3.1.1 Complexity Analysis 21
3.2 Flash Attention (Dao et al., 2022) 21

3.2.1 The Memory Wall Problem . . . 21
3.2.2 Core Insight: Tiling + Online Softmax . . . 21
3.2.3 Flash Attention Algorithm . . . 22
3.2.4 Flash Attention-2 (Dao, 2023) . . . 22
3.2.5 Flash Attention-3 (Hopper Architecture, 2024) . . . 23
3.2.6 Practical Usage . . . 23
3.3 Paged Attention (vLLM, Kwon et al., 2023) . . . 23
3.3.1 Problem: KV-Cache Fragmentation . . . 23
3.3.2 Solution: Virtual Memory for Attention . . . 24
3.3.3 Attention Kernel Design . . . 24
3.3.4 Impact . . . 24
3.4 Sliding Window Attention . . . 24
3.4.1 Definition . . . 24
3.4.2 Multi-Scale Sliding Window (Longformer, Beltagy et al., 2020) . . . 25
3.4.3 Mistral's Sliding Window . . . 25
3.5 Ring Attention (Liu et al., 2023) . . . 25
3.5.1 Motivation . . . 25
3.5.2 Algorithm . . . 25
3.5.3 Striped Attention (Brandon et al., 2023) . . . 25
3.6 Other Efficient Attention Variants . . . 25
3.6.1 Linear (Kernelized) Attention . . . 25
3.6.2 Block Sparse Attention . . . 26
3.6.3 Dilated / Strided Attention (Sparse Transformer) . . . 26
3.6.4 Cross-Layer Attention Sharing . . . 26
3.6.5 Infini-Attention (Google, 2024) . . . 26
3.6.6 Landmark Attention . . . 27
3.7 Attention Optimization Summary . . . 27

4 State Space Models and Mamba 29
4.1 Continuous-Time SSMs . . . 29
4.1.1 State Space Representation . . . 29
4.1.2 Discretization . . . 29
4.1.3 Bilinear (Tustin) Discretization . . . 29
4.2 S4: Structured State Space Sequence Model (Gu et al., 2022) . . . 30
4.2.1 The HiPPO Matrix . . . 30
4.2.2 Normal Plus Low-Rank (NPLR) Decomposition . . . 30
4.2.3 Convolutional View . . . 30
4.3 S5 and S6: Towards Mamba . . . 30
4.3.1 S5: Multi-Input Multi-Output SSM (Smith et al., 2023) . . . 30
4.3.2 S6 / Mamba: Selective SSM (Gu & Dao, 2023) . . . 31
4.3.3 Mamba Block Structure . . . 31
4.4 Mamba-2: Structured State Space Duality (Dao & Gu, 2024) . . . 32
4.4.1 SSM-Attention Duality . . . 32
4.4.2 Key Improvements over Mamba-1 . . . 32
4.5 RWKV: Receptance Weighted Key Value (Peng et al., 2023) . . . 32
4.6 Other SSM Architectures . . . 33
4.6.1 H3: Hungry Hungry Hippos (Fu et al., 2023) . . . 33
4.6.2 Hyena (Poli et al., 2023) . . . 33

4.6.3 Griffin (De et al., 2024) . . . 33
4.6.4 Jamba (AI21 Labs, 2024) . . . 33
4.7 Comparing SSM Architectures . . . 33
4.8 Mathematical Intuition: Why SSMs Work for Language . . . 34

5 Quantization: Theory and Practice **35**
5.1 Quantization Fundamentals . . . 35
5.1.1 Uniform Affine Quantization . . . 35
5.1.2 Scale Determination Strategies . . . 35
5.1.3 Quantization Granularity . . . 35
5.1.4 Quantization-Aware Training (QAT) . . . 36
5.1.5 Post-Training Quantization (PTQ) . . . 36
5.2 LLM.int8() — Mixed-Precision Quantization (Dettmers et al., 2022) . . . 36
5.2.1 The Outlier Problem . . . 36
5.2.2 LLM.int8() Algorithm . . . 36
5.3 GPTQ: Accurate Post-Training Quantization (Frantar et al., 2023) . . . 37
5.3.1 Optimal Brain Quantization (OBQ) Foundation . . . 37
5.3.2 The OBQ Iterative Algorithm . . . 37
5.3.3 GPTQ Optimizations . . . 37
5.3.4 GPTQ Practical Usage . . . 37
5.4 AWQ: Activation-Aware Weight Quantization (Lin et al., 2024) . . . 38
5.4.1 Core Insight . . . 38
5.4.2 Per-Channel Scaling Transformation . . . 38
5.4.3 Algorithm . . . 38
5.5 GGUF / GGML: CPU-Centric Quantization . . . 38
5.5.1 The GGML Ecosystem . . . 38
5.5.2 Quantization Types . . . 39
5.6 NF4: NormalFloat 4-bit (Dettmers et al., 2023) . . . 39
5.6.1 Information-Theoretic Quantization . . . 39
5.6.2 Block-wise Implementation . . . 40
5.7 HQQ: Half-Quadratic Quantization (Badri & Shaji, 2023) . . . 40
5.7.1 Formulation as Optimization Problem . . . 40
5.8 QUIP and QuIP# (Chee et al., 2024) . . . 40
5.8.1 Incoherence Processing . . . 40
5.8.2 QuIP# Improvements . . . 40
5.9 SmoothQuant (Xiao et al., 2023) . . . 41
5.9.1 Activation-Weight Migration . . . 41
5.10 Additional Quantization Methods . . . 41
5.10.1 AQLM (Additive Quantization, 2024) . . . 41
5.10.2 SpQR (Sparse-Quantized Representation, 2023) . . . 41
5.10.3 FP8 / E4M3 / E5M2 . . . 41
5.10.4 MX-FP (Microscaling, 2024) . . . 41
5.10.5 FlexRound (2024) . . . 41
5.10.6 Binary/Ternary Quantization . . . 42
5.11 Comparison of PTQ Methods . . . 42
5.12 Practical Quantization Guide . . . 42

6 Pruning and Sparsity **43**

6.1 Unstructured (Magnitude) Pruning . . . 43
6.1.1 Definition . . . 43
6.1.2 Sparsity Schedules . . . 43
6.1.3 The Lottery Ticket Hypothesis . . . 43
6.1.4 Movement Pruning (Sanh et al., 2020) . . . 43
6.2 SparseGPT (Frantar & Alistarh, 2023) . . . 44
6.2.1 Motivation . . . 44
6.2.2 Algorithm: Extending OBQ to Pruning . . . 44
6.2.3 Performance . . . 44
6.3 Wanda: Pruning by Weights AND Activations (Sun et al., 2024) . . . 44
6.3.1 Core Idea . . . 44
6.3.2 Comparison with SparseGPT . . . 45
6.4 Structured Pruning . . . 45
6.4.1 Motivation . . . 45
6.4.2 Types . . . 45
6.4.3 Importance Estimation for Structured Pruning . . . 45
6.5 N:M Sparsity and Hardware . . . 45
6.5.1 NVIDIA 2:4 Pattern . . . 45
6.6 DSnoT: Dynamic Sparse No-Training Pruning (2024) . . . 46
6.7 SliceGPT (Ashkboos et al., 2024) . . . 46
6.8 Pruning + Quantization Synergy . . . 46

7 Knowledge Distillation **47**
7.1 Foundational Distillation (Hinton et al., 2015) . . . 47
7.1.1 Soft Targets and Temperature . . . 47
7.1.2 Why Temperature Helps . . . 47
7.2 Logit-Based Distillation Variants . . . 47
7.2.1 Standard KD (Soft Targets) . . . 47
7.2.2 DistilBERT (Sanh et al., 2019) . . . 48
7.2.3 TinyBERT (Jiao et al., 2020) . . . 48
7.2.4 MiniLLM (Gu et al., 2024) . . . 48
7.3 Feature-Based Distillation . . . 48
7.3.1 Hidden State Matching . . . 48
7.3.2 Attention Transfer (Zagoruyko & Komodakis, 2017) . . . 48
7.3.3 Relation-Based Distillation . . . 48
7.3.4 FitNet (Romero et al., 2015) . . . 49
7.4 Multi-Teacher and Ensemble Distillation . . . 49
7.4.1 Multi-Teacher Distillation . . . 49
7.4.2 Ensemble Distillation (Born-Again Networks) . . . 49
7.5 Data-Free Distillation . . . 49
7.6 LLM-Specific Distillation Methods . . . 49
7.6.1 Sequence-Level KD (SeqKD) . . . 49
7.6.2 Chat Distillation (Orca / Orca-2, 2023–2024) . . . 50
7.6.3 In-Context Distillation . . . 50
7.6.4 Progressive Module Distillation . . . 50
7.6.5 LaMini-LM (Wu et al., 2023) . . . 50
7.6.6 MoE Distillation (DeepSeek-V2 style) . . . 50
7.7 Quantization-Aware Distillation . . . 50

8 Parameter-Efficient Fine-Tuning (PEFT) 53
8.1 LoRA: Low-Rank Adaptation (Hu et al., 2022) . . . 53
8.1.1 The Low-Rank Hypothesis . . . 53
8.1.2 Forward Pass . . . 53
8.1.3 Initialization . . . 53
8.1.4 Target Modules . . . 53
8.1.5 LoRA Code . . . 54
8.1.6 LoRA Variants . . . 54
8.2 QLoRA: Quantized LoRA (Dettmers et al., 2023) . . . 54
8.2.1 Enabling 65B Fine-Tuning on 48GB . . . 54
8.2.2 Memory Analysis (7B model) . . . 54
8.3 AdaLoRA: Adaptive Rank Allocation (Zhang et al., 2023) . . . 55
8.3.1 Limitation of Fixed-Rank LoRA . . . 55
8.3.2 SVD-Based Parameterization . . . 55
8.4 DoRA: Weight-Decomposed Low-Rank Adaptation (Liu et al., 2024) . . . 55
8.4.1 Learning Magnitude and Direction Separately . . . 55
8.5 VeRA: Vector-based Random Matrix Adaptation (Kopiczko et al., 2024) 56
8.6 $(IA)^3$: Infused Adapter (Liu et al., 2022) . . . 56
8.6.1 Rescaling Vectors . . . 56
8.7 Prefix Tuning and Prompt Tuning . . . 56
8.7.1 Prefix Tuning (Li & Liang, 2021) . . . 56
8.7.2 Prompt Tuning (Lester et al., 2021) . . . 56
8.7.3 P-Tuning v2 . . . 57
8.8 Additional PEFT Variants . . . 57
8.8.1 LoHA (Low-Rank Hadamard Product) . . . 57
8.8.2 LoKr (Low-Rank Kronecker Product) . . . 57
8.8.3 OFT / BOFT (Orthogonal Fine-Tuning) . . . 57
8.8.4 SSF (Scaling and Shifting Features) . . . 57
8.8.5 KronA (Kronecker Adapter) . . . 57
8.8.6 MoRA (Mixture of Rank-1 Adapters) . . . 57
8.9 PEFT Comparison . . . 58
8.10 PEFT Best Practices . . . 58

9 Preference Alignment Techniques 59
9.1 RLHF: Reinforcement Learning from Human Feedback . . . 59
9.1.1 Three-Stage Pipeline (Christiano et al., 2017; Ouyang et al., 2022) 59
9.1.2 PPO Mechanic . . . 59
9.1.3 RLHF Variants . . . 60
9.2 DPO: Direct Preference Optimization (Rafailov et al., 2023) . . . 60
9.2.1 Eliminating the Reward Model . . . 60
9.2.2 Gradient Analysis . . . 60
9.2.3 DPO Variants . . . 60
9.3 IPO: Identity Preference Optimization (Azar et al., 2024) . . . 61
9.3.1 Addressing DPO Limitations . . . 61
9.4 KTO: Kahneman-Tversky Optimization (Ethayarajh et al., 2024) . . . 61
9.4.1 Prospect Theory Motivation . . . 61
9.4.2 No Paired Data Required . . . 61
9.5 ORPO: Odds Ratio Preference Optimization (Hong et al., 2024) . . . 62

9.5.1 Unified SFT + Alignment . . . 62
9.6 SimPO: Simple Preference Optimization (Meng et al., 2024) . . . 62
9.6.1 Reference-Free with Length Normalization . . . 62
9.7 RLAIF: RL from AI Feedback (Bai et al., 2022) . . . 62
9.7.1 Constitutional AI . . . 62
9.7.2 UltraFeedback (Cui et al., 2023) . . . 63
9.8 RLVR: RL with Verifiable Rewards (Lambert et al., 2024) . . . 63
9.8.1 Outcome-Based Binary Rewards . . . 63
9.9 CPO: Contrastive Preference Optimization (Xu et al., 2024) . . . 63
9.10 SPIN: Self-Play Fine-Tuning (Chen et al., 2024) . . . 63
9.11 Comparison of Alignment Methods . . . 64
9.12 Practical Recommendations . . . 64

10 Mixture of Experts (MoE) 65
10.1 MoE Architecture Fundamentals . . . 65
10.1.1 Core Idea . . . 65
10.1.2 Load Balancing . . . 65
10.1.3 Expert Capacity . . . 66
10.2 Major MoE Implementations . . . 66
10.2.1 Switch Transformer (Fedus et al., 2022) . . . 66
10.2.2 GShard (Lepikhin et al., 2021) . . . 66
10.2.3 Mixtral 8×7B (Jiang et al., 2024) . . . 66
10.2.4 DeepSeek-MoE (Dai et al., 2024) . . . 66
10.2.5 Qwen2-MoE / dMoE . . . 66
10.2.6 OpenMoE / JetMoE . . . 66
10.3 Expert Routing Strategies . . . 67
10.3.1 Token-Choice Routing (Standard) . . . 67
10.3.2 Expert-Choice Routing (Zhou et al., 2022) . . . 67
10.3.3 Hash Routing (Roller et al., 2021) . . . 67
10.3.4 Soft MoE (Puigcerver et al., 2023) . . . 67
10.3.5 DeepSeek-V2 Routing . . . 67
10.4 MoE for Attention (MoA) . . . 67
10.5 PEER: Parameter Efficient Expert Retrieval (He et al., 2024) . . . 68
10.6 MoE Inference Considerations . . . 68
10.6.1 Memory Requirements . . . 68
10.6.2 Expert Parallelism Communication . . . 68
10.6.3 Expert Caching . . . 68
10.7 MoE Training Stability Techniques . . . 68

11 Inference Engine Selection and Hardware Optimization 71
11.1 Major Inference Engines . . . 71
11.1.1 llama.cpp (CPU, Edge, and GPU via BLAS) . . . 71
11.1.2 vLLM (GPU Server, Kwon et al., 2023) . . . 71
11.1.3 Text Generation Inference (TGI, HuggingFace) . . . 72
11.1.4 TensorRT-LLM (NVIDIA) . . . 72
11.1.5 SGLang (Structured Generation Language) . . . 72
11.1.6 MLX (Apple Silicon) . . . 72
11.1.7 Ollama . . . 73

11.1.8 Other Notable Engines . . . 73
11.2 Hardware-Specific Optimizations . . . 73
11.2.1 CUDA Kernel Optimization . . . 73
11.2.2 CPU Optimization . . . 74
11.2.3 GPU Memory Optimization . . . 74
11.3 Multi-GPU Deployment Strategies . . . 74
11.3.1 Tensor Parallelism (TP) . . . 74
11.3.2 Pipeline Parallelism (PP) . . . 74
11.3.3 Data Parallelism (DP) for Inference . . . 74
11.3.4 ZeRO-Inference (DeepSpeed) . . . 75
11.3.5 Sequence Parallelism . . . 75
11.4 Offloading Techniques . . . 75
11.4.1 CPU Offloading . . . 75
11.4.2 Disk Offloading (FlexGen, Sheng et al., 2023) . . . 75
11.4.3 Attention Offloading (AOffload) . . . 75
11.5 Batching and Scheduling . . . 75
11.5.1 Continuous (In-flight) Batching . . . 75
11.5.2 Chunked Prefill (Sarathi/Splitwise) . . . 76
11.5.3 Priority Scheduling . . . 76
11.6 Inference Engine Selection Guide . . . 76

12 Decoding Techniques **77**
12.1 Deterministic Decoding . . . 77
12.1.1 Greedy Decoding . . . 77
12.1.2 Beam Search . . . 77
12.1.3 Diverse Beam Search (Vijayakumar et al., 2018) . . . 77
12.1.4 Contrastive Search (Su et al., 2022) . . . 77
12.2 Sampling-Based Decoding . . . 78
12.2.1 Temperature Sampling . . . 78
12.2.2 Top-k Sampling (Fan et al., 2018) . . . 78
12.2.3 Nucleus (Top-p) Sampling (Holtzman et al., 2020) . . . 78
12.2.4 Min-p Sampling (Nguyen et al., 2024) . . . 78
12.2.5 Typical Sampling (Meister et al., 2023) . . . 78
12.2.6 Eta (η) Sampling (Hewitt et al., 2022) . . . 79
12.2.7 Repetition Penalties . . . 79
12.2.8 Mirostat (Basu et al., 2021) . . . 79
12.3 Constrained and Guided Decoding . . . 79
12.3.1 Formal Language Constraints . . . 79
12.3.2 Constrained Decoding with Outlines . . . 79
12.3.3 Classifier-Free Guidance (Sanchez et al., 2023) . . . 80
12.4 Speculative Decoding . . . 80
12.4.1 Basic Speculative Decoding (Leviathan et al., 2023) . . . 80
12.4.2 Medusa (Cai et al., 2024) . . . 80
12.4.3 Eagle (Li et al., 2024) . . . 80
12.4.4 Self-Speculative Decoding . . . 80
12.4.5 Variants and Optimizations . . . 81
12.5 Advanced Sampling and Generation . . . 81
12.5.1 Locally Typical Sampling (Meister et al., 2022) . . . 81

12.5.2 DoLa (Decoding by Contrasting Layers, Chuang et al., 2023) . . . 81
12.5.3 Expert Iteration / STaR (Zelikman et al., 2022) . . . 81
12.5.4 Consistency Decoding (CD) . . . 81
12.5.5 Entropy-Based Decoding . . . 81
12.5.6 Batched Speculative Decoding . . . 81

13 Retrieval-Augmented Generation (RAG) 83
13.1 RAG Fundamentals . . . 83
13.1.1 Naive RAG Pipeline (Lewis et al., 2020) . . . 83
13.2 Advanced RAG Architectures . . . 83
13.2.1 Modular RAG (Gao et al., 2023) . . . 83
13.2.2 Hybrid RAG (Dense + Sparse) . . . 84
13.2.3 Hierarchical RAG (Multi-Level Indexing) . . . 84
13.2.4 Self-RAG (Asai et al., 2023) . . . 84
13.2.5 Corrective RAG (CRAG, Yan et al., 2024) . . . 84
13.3 ColBERT and Late Interaction Retrieval . . . 84
13.3.1 ColBERT v1/v2 (Khattab & Zaharia, 2020) . . . 84
13.3.2 SPLADE (Sparse Lexical and Expansion) . . . 85
13.3.3 PLAID (Efficient ColBERT) . . . 85
13.4 Retrieval Optimizations . . . 85
13.4.1 Re-ranking with Cross-Encoders . . . 85
13.4.2 Query Transformations . . . 85
13.4.3 Context Compression (LLMLingua) . . . 86
13.4.4 Fusion / Reciprocal Rank Fusion (RRF) . . . 86
13.5 Embedding Models for RAG . . . 86
13.5.1 Bi-Encoder Architectures . . . 86
13.5.2 Embedding Tuning and Matryoshka Embeddings . . . 86
13.6 Multi-Hop and Agentic RAG . . . 86
13.6.1 Multi-Hop Retrieval . . . 86
13.6.2 Agentic RAG (ARAG) . . . 87
13.6.3 Graph RAG (Microsoft, 2024) . . . 87
13.7 Multimodal RAG . . . 87
13.8 RAG Evaluation Metrics . . . 87
13.9 Caching Strategies for RAG . . . 88

14 Hooking, Model Interpretability, and Scheduling 89
14.1 Hooking: Intercepting Model Internals . . . 89
14.1.1 PyTorch Forward Hooks . . . 89
14.1.2 TransformerLens (Nanda & Bloom, 2022) . . . 89
14.1.3 NNsight (Jaden, 2024) . . . 90
14.1.4 Steering Vectors / Activation Engineering . . . 90
14.1.5 Representation Engineering (RepE, Zou et al., 2023) . . . 90
14.1.6 Refusal Vector Ablation (Arditi et al., 2024) . . . 90
14.1.7 Logit Lens and Tuned Lens . . . 91
14.1.8 SAE (Sparse Autoencoder) for Feature Decomposition . . . 91
14.1.9 Patching and Causal Tracing . . . 91
14.2 Scheduling and Batching Strategies . . . 91
14.2.1 Static Batching . . . 91

14.2.2 Continuous Batching (Iteration-Level Scheduling) 91
14.2.3 Chunked Prefill Scheduling 92
14.2.4 Priority-Based Scheduling 92
14.2.5 Request Scheduling for LoRA Serving 92
14.2.6 Power Management and Thermal Throttling 92
14.2.7 Speculative Scheduling 93

15 Domain Expert Small LLMs **95**
15.1 Major Small LLM Families 95
15.1.1 LLaMA Family (Meta) 95
15.1.2 Mistral (Mistral AI) 95
15.1.3 Qwen Series (Alibaba) 95
15.1.4 Gemma (Google) 96
15.1.5 Phi Series (Microsoft) 96
15.1.6 DeepSeek 96
15.1.7 Additional Notable Small LLMs 96
15.2 Domain Specialization Strategies for Small LLMs 96
15.2.1 Continued Pre-Training (CPT) 96
15.2.2 Instruction Fine-Tuning for Domain Tasks 97
15.2.3 Retrieval-Augmented Domain Models 97
15.2.4 Ensemble and Specialization Routing 97
15.3 Training Recipes for Small LLMs 98
15.3.1 Data Quality and Curation 98
15.3.2 Training Efficiency Techniques 98
15.3.3 LR Schedules and Hyperparameters 98
15.3.4 Chinchilla-Optimal vs Over-Training 98
15.4 Locally Deployable Domain Expert Architecture 99
15.4.1 Quantized Domain Model Stack 99
15.4.2 Edge Deployment (Raspberry Pi / Mobile) 99
15.4.3 Local RAG + LLM Stack 99
15.5 Future Directions for Small LLMs 100
15.6 Small LLM Selection Guide 100

Chapter 1

Introduction

Running large language models (LLMs) on consumer hardware—laptops, desktops, even embedded devices—has become one of the most exciting frontiers in applied machine learning. This book is a comprehensive technical reference covering every major technique that makes local LLM deployment possible: from the architectural innovations that make models efficient, through quantization, pruning, and distillation methods that shrink them, to inference engines and decoding strategies that make them fast.

Who this book is for: Practitioners who want to understand not just *what* techniques exist, but *why* they work mathematically and *how* to implement them. Every chapter provides the mathematical principles, algorithmic details, relevant code examples, and practical trade-offs.

Prerequisites: Familiarity with linear algebra, probability theory, and the transformer architecture. We review essentials where needed.

Chapter 2

Efficient Base Model Architecture

Modern efficient LLMs share a common architectural DNA, most famously documented by the LLaMA family (Touvron et al., 2023). This chapter dissects each architectural component, explaining both the original motivation and the mathematical principles.

2.1 RMSNorm (Root Mean Square Layer Normalization)

2.1.1 Motivation

LayerNorm (Ba et al., 2016) is expensive: it computes both mean μ and variance σ^2 over the hidden dimension. RMSNorm (Zhang & Sennrich, 2019) drops the mean-centering step, reducing compute by roughly 30% while maintaining training stability.

2.1.2 Mathematical Definition

Classical LayerNorm for an input vector $\mathbf{x} \in \mathbb{R}^d$:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 \tag{2.1}$$

$$\text{LayerNorm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta} \tag{2.2}$$

RMSNorm simplifies this significantly:

$$\text{RMSNorm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \tag{2.3}$$

where $\boldsymbol{\gamma} \in \mathbb{R}^d$ is a learnable gain parameter. Note there is no bias $\boldsymbol{\beta}$ and no mean subtraction. The denominator $\sqrt{\frac{1}{d}\sum x_i^2}$ is the root mean square.

2.1.3 Why It Works

Zhang & Sennrich (2019) proved that the mean-centering step is not critical for re-centering invariance, since weight matrices and subsequent layers can absorb the offset.

Empirically, RMSNorm achieves identical convergence to LayerNorm with 10–30% faster training.

2.1.4 Implementation

Listing 2.1: RMSNorm in PyTorch

```
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x):
        # x shape: (..., dim)
        rms = torch.sqrt(x.pow(2).mean(-1, keepdim=True) + self.
            eps)
        return x / rms * self.weight
```

2.2 SwiGLU Activation

2.2.1 Background: Gated Linear Units

The Gated Linear Unit (GLU; Dauphin et al., 2017) splits the output of a linear transform and applies a gating non-linearity:

$$\mathrm{GLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \odot \sigma(\mathbf{x}\mathbf{W}_2 + \mathbf{b}_2) \tag{2.4}$$

where σ is typically the sigmoid function.

2.2.2 SwiGLU Definition

Shazeer (2020) explored replacing σ with the Swish (SiLU) activation, producing the **SwiGLU**:

$$\mathrm{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_1) \odot \mathrm{SiLU}(\mathbf{x}\mathbf{W}_2) \tag{2.5}$$

SiLU is defined as $\mathrm{SiLU}(x) = x \cdot \sigma(x)$. For the feed-forward network (FFN), the full SwiGLU FFN becomes:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}) = \big(\mathrm{SiLU}(\mathbf{x}\mathbf{W}_{\mathrm{gate}}) \odot \mathbf{x}\mathbf{W}_{\mathrm{up}}\big)\mathbf{W}_{\mathrm{down}} \tag{2.6}$$

2.2.3 Why SwiGLU Outperforms ReLU/GELU

1. **Non-monotonic bump:** SiLU has a small negative region near $x \approx -1.28$, allowing selective dampening of activations rather than just zeroing them (ReLU) or scaling them (GELU).

2. **Smooth gradient:** SiLU is everywhere differentiable with $\text{SiLU}'(x) = \sigma(x) + x\sigma(x)(1-\sigma(x))$, avoiding the dead-neuron problem of ReLU.
3. **Gating mechanism:** The multiplicative interaction enables the network to learn sharper decision boundaries through the gate-up product.

2.2.4 Variants of Gated Activations

- **GeGLU:** Uses GELU gating instead of SiLU. Empirically slightly worse than SwiGLU but still better than standard FFN.
- **ReGLU:** Uses ReLU gating. Faster to compute but less expressive.
- **Bilinear GLU:** $(\mathbf{x}\mathbf{W}_1) \cdot \sigma((\mathbf{x}\mathbf{W}_2) \odot (\mathbf{x}\mathbf{W}_3))$ — adds a second gating signal.
- **Squared ReLU (dReLU):** Used in Gemma architecture: $\text{FFN}(\mathbf{x}) = \mathbf{W}_{\text{down}} \cdot \text{ReLU}(\mathbf{W}_{\text{gate}}\mathbf{x})^2 \odot \mathbf{W}_{\text{up}}\mathbf{x}$.
- **Combined activation:** $\text{SiLU}(x) + \epsilon \cdot \text{ReLU}(x)$ as a tunable hybrid.

2.2.5 Weight Scaling

SwiGLU has three weight matrices (gate, up, down) vs. two in a standard FFN. To keep parameter counts comparable, the hidden dimension is scaled: if standard FFN uses $4d_{\text{model}}$, the SwiGLU FFN uses $\frac{2}{3} \cdot 4d_{\text{model}} = \frac{8}{3}d_{\text{model}}$, maintaining roughly the same total parameters.

2.3 RoPE: Rotary Positional Embeddings

2.3.1 Problem With Absolute Positional Encodings

Sinusoidal absolute position encodings (Vaswani et al., 2017) add position-dependent vectors to token embeddings. Learned absolute embeddings consume vocabulary-head parameters. Both approaches encode position in the input space rather than the attention space, limiting their ability to capture *relative* distances between tokens.

2.3.2 Mathematical Foundation: Rotation in 2D

RoPE (Su et al., 2024) encodes position by rotating query and key vectors in 2D subspaces. The core insight: a rotation in $\mathbb{R}^2$ by angle θ preserves dot product magnitude, and the dot product of two vectors rotated by θ_1, θ_2 depends only on the rotation difference:

$$\langle R_{\theta_1}\mathbf{q}, R_{\theta_2}\mathbf{k}\rangle = \mathbf{q}^\top R_{\theta_1}^\top R_{\theta_2}\mathbf{k} = \mathbf{q}^\top R_{\theta_2-\theta_1}\mathbf{k} \tag{2.7}$$

For a d-dimensional vector, consecutive dimensions (x_{2i}, x_{2i+1}) are paired and each pair is rotated with a frequency that decays with dimension index:

$$\mathbf{x}' = \big(x_0\cos(m\theta_0) - x_1\sin(m\theta_0),\ x_0\sin(m\theta_0) + x_1\cos(m\theta_0), \tag{2.8}$$

$$x_2\cos(m\theta_1) - x_3\sin(m\theta_1),\ x_2\sin(m\theta_1) + x_3\cos(m\theta_1), \dots\big) \tag{2.9}$$

where m is the token position and $\theta_i = 10000^{-2i/d}$ (the same frequency schedule as sinusoidal encodings).

2.3.3 Efficient Implementation

Rather than constructing rotation matrices (which would be $O(d^2)$), RoPE uses an element-wise transformation:

$$f_{\text{RoPE}}(\mathbf{x}, m) = \mathbf{x} \odot \begin{pmatrix} \cos(m\theta_0) \\ \cos(m\theta_0) \\ \cos(m\theta_1) \\ \cos(m\theta_1) \\ \vdots \\ \cos(m\theta_{d/2-1}) \\ \cos(m\theta_{d/2-1}) \end{pmatrix} + \begin{pmatrix} -x_1 \\ x_0 \\ -x_3 \\ x_2 \\ \vdots \\ -x_{d-1} \\ x_{d-2} \end{pmatrix} \odot \begin{pmatrix} \sin(m\theta_0) \\ \sin(m\theta_0) \\ \sin(m\theta_1) \\ \sin(m\theta_1) \\ \vdots \\ \sin(m\theta_{d/2-1}) \\ \sin(m\theta_{d/2-1}) \end{pmatrix} \tag{2.10}$$

This is $O(d)$ and easily vectorized. In practice, it's implemented using complex number multiplication: treat each pair (x_{2i}, x_{2i+1}) as a complex number and multiply by $e^{im\theta_i}$.

2.3.4 RoPE Variants for Context Length Extension

- **Linear Interpolation:** $\theta_i' = \theta_i \cdot (L_{\text{max}}/L_{\text{new}})$ — simplest method. Rescales all frequencies uniformly. Loses high-frequency resolution, causing "lobotomized" attention for long sequences.

- **NTK-Aware (Neural Tangent Kernel) Interpolation:** Scale high frequencies less than low frequencies (non-uniform scaling):

  $$\theta_i' = \theta_i \cdot s^{-2i/(d-2)}, \quad s = L_{\text{new}}/L_{\text{max}} \tag{2.11}$$

  This preserves the "resolution" of high-frequency components that encode local positional information while stretching low frequencies for long-range positions.

- **YaRN (Yet another RoPE extensioN):** Combines NTK-aware scaling with a temperature-based attention re-weighting. Adds a "length scaling" factor $\sqrt{t}$ to the attention softmax to counteract entropy increase from longer sequences. Also introduces partial NTK-aware scaling only for dimensions above a cutoff.

- **Dynamic NTK:** Adjusts the scale factor s dynamically based on the current sequence length, so the model can handle variable-length inputs up to any length without fixed interpolation.

- **xPos (Sun et al., 2022):** Extends RoPE with an exponentially decaying amplitude envelope $r_i = \gamma^{-2i/d}$ on the key vectors only:

  $$\mathbf{k}' = \boldsymbol{r} \odot \mathbf{k}, \quad r_i = \gamma^{-2i/d} \tag{2.12}$$

  This attenuates distant key vectors, improving length extrapolation.

- **ReRoPE (Rectified RoPE):** Uses piecewise frequency patterns where frequencies from different dimensions are "rectified" (truncated) at different thresholds, preventing high-frequency dimension confusion on long sequences.

- **CLEX (Continuous Length EXtrapolation):** Instead of interpolation, learns a continuous length scaling function $f(\ell, \theta_i)$ that maps arbitrary lengths to effective frequencies through a learned neural ODE.

- **Self-Extend:** Uses floor division for position grouping: $m' = \lfloor m/G \rfloor$ for some dimensions, preserving fine-grained local positions while coarsening distant ones.

2.3.5 PyTorch Implementation (Complex Number Approach)

Listing 2.2: RoPE using complex numbers

```
def precompute_freqs_cis(dim: int, max_seq_len: int, theta: float
    = 10000.0):
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2, dtype=torch.
        float)[:dim//2] / dim))
    t = torch.arange(max_seq_len, dtype=torch.float)
    freqs = torch.outer(t, freqs)  # [max_seq_len, dim//2]
    return torch.polar(torch.ones_like(freqs), freqs)  # e^{i*m*
        theta}

def apply_rotary_emb(xq, xk, freqs_cis):
    # xq, xk: [batch, heads, seq_len, head_dim]
    xq_ = torch.view_as_complex(xq.float().reshape(*xq.shape[:-1],
        -1, 2))
    xk_ = torch.view_as_complex(xk.float().reshape(*xk.shape[:-1],
        -1, 2))
    freqs_cis = freqs_cis[:xq_.shape[-2]]
    # Reshape freqs_cis to broadcast: [1, 1, seq_len, head_dim//2]
    freqs_cis = freqs_cis.unsqueeze(0).unsqueeze(0)
    xq_out = torch.view_as_real(xq_ * freqs_cis).flatten(3)
    xk_out = torch.view_as_real(xk_ * freqs_cis).flatten(3)
    return xq_out.type_as(xq), xk_out.type_as(xk)
```

2.4 Grouped Query Attention (GQA)

2.4.1 The Attention Hierarchy: MHA → MQA → GQA

- **Multi-Head Attention (MHA):** h heads, each with its own $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i$. Full expressivity at maximal KV-cache memory cost: $O(2 \cdot h \cdot d_k \cdot n_{\text{layers}} \cdot L)$.
- **Multi-Query Attention (MQA):** All heads share a single $\mathbf{K}$ and $\mathbf{V}$. Drastically reduces KV-cache but degrades quality for large models.
- **Grouped Query Attention (GQA):** g groups of heads, each group shares one $\mathbf{K}, \mathbf{V}$ pair. Balances memory and quality.

2.4.2 Formal Definition

Given h query heads and g KV groups (with $g \mid h$), each KV head serves h/g query heads. For query head index i:

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_{\lfloor i \cdot g/h \rfloor}, \mathbf{V}_{\lfloor i \cdot g/h \rfloor}) \tag{2.13}$$

GQA generalizes both MHA ($g = h$) and MQA ($g = 1$). LLaMA-2 70B uses $g = 8$, achieving 97%+ of MHA quality while reducing KV-cache memory by 8×.

2.4.3 KV-Cache Memory Comparison

For a L-layer model with sequence length S, hidden dimension d, and per-head dimension d_h:

$$\text{MHA KV-cache} = 2 \cdot L \cdot S \cdot h \cdot d_h = 2 \cdot L \cdot S \cdot d \quad \text{bytes in FP16} \tag{2.14}$$

$$\text{GQA KV-cache} = 2 \cdot L \cdot S \cdot g \cdot d_h = 2 \cdot L \cdot S \cdot g \cdot d/h \quad \text{bytes} \tag{2.15}$$

For LLaMA-2 70B ($d = 8192$, $h = 64$, $g = 8$), a 2048-token sequence requires only ∼164 MB for GQA vs ∼1.3 GB for MHA.

2.4.4 Training Considerations

GQA can be obtained by:

1. **Training from scratch:** Straightforward but computationally expensive.
2. **Uptraining from MHA checkpoint:** Mean-pool the pre-trained $\mathbf{K}_i, \mathbf{V}_i$ heads within each group to initialize shared heads, then continue pre-training for ∼5–10% of original steps. Converges quickly since the initialization is close to the MHA behavior.
3. **Direct conversion:** For small models, direct mean-pooling without uptraining sometimes suffices in practice, though quality degrades measurably for models over 7B.

2.4.5 Relationship to Other KV-Cache Reduction Methods

- **KV-cache quantization:** Store cached K and V values in INT8 or FP8 instead of FP16. Orthogonal to GQA—both can be applied simultaneously for multiplicative savings.
- **Sliding Window + GQA:** Mistral combines sliding window attention with GQA for extreme memory efficiency with long contexts.
- **Layer-wise KV sharing:** Share KV-cache across layers (not just heads within a layer). An extreme variant of cross-layer attention.

2.5 Tied Embeddings and Shared Layers

2.5.1 Input-Output Embedding Tying (Weight Tying)

In standard transformer LMs, the input embedding matrix $\mathbf{E}_{\text{in}} \in \mathbb{R}^{V \times d}$ maps token IDs to hidden vectors, and the output projection $\mathbf{E}_{\text{out}} \in \mathbb{R}^{d \times V}$ maps hidden states to logits. **Weight tying** sets $\mathbf{E}_{\text{out}} = \mathbf{E}_{\text{in}}^\top$.

Mathematical justification: If words w_1, w_2 have similar embeddings $\mathbf{e}_1 \approx \mathbf{e}_2$, then for any hidden state $\mathbf{h}$, the logits $\mathbf{h}^\top \mathbf{e}_1 \approx \mathbf{h}^\top \mathbf{e}_2$ will be similar—the model should predict them similarly. This inductive bias reduces parameters by $\approx d \times V$ (often 20–40% of total) with minimal perplexity degradation.

Advanced variants:

- **Three-way tying:** Tie embeddings between input, output, AND a separate "softmax" embedding.
- **Projected tying:** $\mathbf{E}_{\text{out}} = \mathbf{P}\mathbf{E}_{\text{in}}^\top$ where $\mathbf{P} \in \mathbb{R}^{V \times V}$ is a low-rank projection.

- **Dual tying:** Share embeddings only for a subset of the vocabulary (e.g., the most frequent tokens).

2.5.2 Cross-Layer Parameter Sharing (ALBERT-style)

ALBERT (Lan et al., 2019) demonstrated that all transformer layers can *share the same parameters*. The forward pass becomes:

$$\mathbf{H}^{(\ell+1)} = f_\theta(\mathbf{H}^{(\ell)}), \quad \ell = 0, 1, \ldots, L-1 \tag{2.16}$$

where f_θ is the same function applied repeatedly. This reduces parameters by factor L at the cost of expressivity—the model must learn a function useful at every depth simultaneously.

Modern variants:

- **Block-wise sharing:** Group layers into blocks (e.g., every 4 layers share parameters). Each block learns a distinct function but parameters are reused within it.
- **Stochastic depth:** Randomly skip layers during training, encouraging layer independence so some can be dropped at inference.
- **Funnel transformers:** Gradually compress sequence length across layers while sharing parameters within each compression level.
- **Rerope Layer Sharing:** Share attention weights but use different RoPE frequencies per layer to distinguish layer functionality.

2.6 Other Architecture Optimizations

2.6.1 Parallel Attention/FFN (PaLM-style)

Instead of the sequential Attention → FFN in standard transformers, PaLM (Chowdhery et al., 2022) computes them in parallel:

$$\mathbf{H}^{(\ell+1)} = \mathbf{H}^{(\ell)} + \text{Attn}(\text{RMSNorm}(\mathbf{H}^{(\ell)})) + \text{FFN}(\text{RMSNorm}(\mathbf{H}^{(\ell)})) \tag{2.17}$$

This enables more parallel computation and was found to improve training stability at scale with reduced communication in model parallelism.

2.6.2 Pre-Norm vs Post-Norm

Post-norm (original Transformer): $\mathbf{H}^{(\ell+1)} = \text{LayerNorm}(\mathbf{H}^{(\ell)} + \text{SubLayer}(\mathbf{H}^{(\ell)}))$
Pre-norm (modern LLMs): $\mathbf{H}^{(\ell+1)} = \mathbf{H}^{(\ell)} + \text{SubLayer}(\text{LayerNorm}(\mathbf{H}^{(\ell)}))$

Pre-norm provides better gradient flow through residual connections, enabling training of much deeper models (100+ layers) without learning rate warmup tricks.

2.6.3 Untied Pre-Norm Gains

Instead of a single $\boldsymbol{\gamma}$ in RMSNorm before attention AND FFN, use separate gains $\boldsymbol{\gamma}_{\text{attn}}$ and $\boldsymbol{\gamma}_{\text{ffn}}$. This small capacity increase (one extra vector per layer) slightly improves expressivity.

2.6.4 Summary of Efficient Architecture Components

Table 2.1: Efficient architecture components and their primary benefits

Component	**Replaces**	**Primary Benefit**
RMSNorm	LayerNorm	10–30% faster normalization
SwiGLU	ReLU/GELU FFN	Better gradient flow, gating
RoPE	Absolute pos. embeddings	Better length generalization
GQA	MHA	Up to 8× KV-cache reduction
Tied embeddings	Separate in/out matrices	20–40% parameter reduction
Shared layers	Distinct per-layer weights	$L\times$ parameter reduction
Pre-norm	Post-norm	Stable deep network training
Parallel Attn/FFN	Sequential blocks	15% faster training at scale

Chapter 3

Attention Mechanisms: A Complete Taxonomy

Attention is the computational core of the transformer. This chapter covers the full spectrum from basic attention to specialized variants enabling local deployment.

3.1 Standard Scaled Dot-Product Attention

Given queries $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{m \times d_k}$, values $\mathbf{V} \in \mathbb{R}^{m \times d_v}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{3.1}$$

The $\sqrt{d_k}$ scaling prevents dot products from growing too large as dimensionality increases, which would push softmax into regions of near-zero gradients.

3.1.1 Complexity Analysis

- **Compute:** $O(n \cdot m \cdot d_k)$ for the attention matrix, $O(n \cdot m \cdot d_v)$ for the output.
- **Memory:** The attention matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ requires $O(n \cdot m)$ memory. For $n = m = 2048$, this is 4M entries (16 MB in FP32) per head per layer—the primary bottleneck for long sequences.

3.2 Flash Attention (Dao et al., 2022)

3.2.1 The Memory Wall Problem

Standard attention materializes the $n \times n$ attention matrix in HBM (GPU global memory), then reads it back for softmax and V multiplication. For large n, HBM bandwidth becomes the bottleneck—the GPU's compute units idle while waiting for data.

3.2.2 Core Insight: Tiling + Online Softmax

Flash Attention keeps computations in SRAM (on-chip) using two key ideas:

1. Tiling: Split $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks fitting in SRAM (typically 128×128 or 64×64). Process attention block-by-block.

2. Online (running) softmax: Standard softmax requires two passes—one for max and sum, one for normalization. The online algorithm maintains running statistics to compute softmax incrementally.

For block-wise computation with blocks $\mathbf{S}^{(1)}, \mathbf{S}^{(2)}$:

$$m^{(1)} = \max(\mathbf{S}^{(1)}), \quad \ell^{(1)} = \sum \exp(S_i^{(1)} - m^{(1)}) \tag{3.2}$$

$$m^{(2)} = \max(m^{(1)}, \max(\mathbf{S}^{(2)})) \tag{3.3}$$

$$\ell^{(2)} = \ell^{(1)} \cdot e^{m^{(1)} - m^{(2)}} + \sum \exp(S_i^{(2)} - m^{(2)}) \tag{3.4}$$

The output is recomputed using the rescaling factor $e^{m^{(1)} - m^{(2)}}$ to correct previously computed partial outputs.

3.2.3 Flash Attention Algorithm

Listing 3.1: Conceptual Flash Attention forward (single head, simplified)

```
def flash_attention(Q, K, V, block_size=128):
    n, d_k = Q.shape
    O = torch.zeros_like(Q)
    L = torch.zeros(n, 1)  # running denominator (row sum)
    M = torch.full((n, 1), -float('inf'))  # running max

    for j in range(0, n, block_size):
        Kj, Vj = K[j:j+block_size], V[j:j+block_size]
        for i in range(0, n, block_size):
            Qi = Q[i:i+block_size]
            S = Qi @ Kj.T  # [block, block] -- in SRAM
            m_new = torch.max(S, dim=-1, keepdim=True).values
            P = torch.exp(S - m_new)  # unnormalized softmax
            l_new = P.sum(dim=-1, keepdim=True)

            m_prev = M[i:i+block_size]
            l_prev = L[i:i+block_size]
            m_max = torch.maximum(m_prev, m_new)
            # Correction factor for existing accumulators
            alpha = torch.exp(m_prev - m_max)
            beta = torch.exp(m_new - m_max)
            l_corr = l_prev * alpha + l_new * beta

            O[i:i+block_size] = (O[i:i+block_size] * l_prev *
                alpha
                                 + P @ Vj) / l_corr
            M[i:i+block_size] = m_max
            L[i:i+block_size] = l_corr
    return O
```

3.2.4 Flash Attention-2 (Dao, 2023)

Improvements over v1:

- **Reduced non-matmul FLOPs:** Restructured algorithm to minimize operations not expressible as GEMMs (which are heavily optimized in CUDA). Rescaling operations are amortized over larger blocks.
- **Parallelism over sequence length:** v2 parallelizes over the sequence-length dimension (in addition to batch and heads), improving GPU occupancy.
- **Better work partitioning:** Each thread block processes one query block against all KV blocks sequentially, reducing inter-thread-block communication and synchronization.
- **Forward-only optimization:** The forward kernel is separated from the backward kernel, allowing independent optimization of each pass.

3.2.5 Flash Attention-3 (Hopper Architecture, 2024)

Exploits NVIDIA H100 (Hopper) hardware features:

- **WGMMA (Warp Group MMA):** Asynchronous matrix multiply-accumulate that can overlap with other operations within the same warp group.
- **TMA (Tensor Memory Accelerator):** Hardware unit for asynchronous 2D memory copy between HBM and shared memory, freeing compute units from address calculation.
- **Asynchronous pipelining:** The forward pass is reformulated as an asynchronous, pipelined computation where data movement, softmax statistics computation, and matrix multiplies execute concurrently.
- **Result:** 1.5–2× speedup over Flash Attention-2 on H100.

3.2.6 Practical Usage

Listing 3.2: Using Flash Attention with PyTorch 2.0+

```
# PyTorch 2.0+ automatically dispatches to Flash Attention
import torch.nn.functional as F
output = F.scaled_dot_product_attention(query, key, value,
    is_causal=True)  # enables auto FA dispatch

# Explicit flash-attn library for more control
from flash_attn import flash_attn_func
output = flash_attn_func(query, key, value, causal=True,
    window_size=(-1, -1))  # -1 = attend to all
```

3.3 Paged Attention (vLLM, Kwon et al., 2023)

3.3.1 Problem: KV-Cache Fragmentation

During autoregressive generation, the KV-cache for each request grows token by token. Naive implementations pre-allocate contiguous memory for the maximum sequence length, leading to:

- **Internal fragmentation:** Unused pre-allocated space within a request's allocation (reserved for potential long sequences).

- **External fragmentation:** Free memory distributed in non-contiguous chunks between allocations.
- **Reservation waste:** Overallocation for the worst-case sequence length.

Combined, memory waste can reach 60–80% in practice, severely limiting batch size.

3.3.2 Solution: Virtual Memory for Attention

PagedAttention maps the KV-cache to non-contiguous **pages** (blocks of tokens, typically 16 tokens each), analogous to virtual memory pages in operating systems. A page table tracks which logical token positions map to which physical KV-cache blocks.

Key operations:

- **Append:** When a new token is generated, write to the current page. If full, allocate a new page from the free pool.
- **Copy-on-write sharing:** During beam search, different beams can share pages for common prefix tokens. When a beam diverges, the page is copied (lazy duplication).
- **Memory reclamation:** When a request finishes, all its pages are freed and returned to the free pool.

3.3.3 Attention Kernel Design

During computation, the kernel iterates over logical blocks and looks up physical locations:

$$\mathbf{o}_t = \sum_{b \in \text{blocks(req)}} \text{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_b^\top}{\sqrt{d_k}}\right) \mathbf{V}_b \tag{3.5}$$

where each $\mathbf{K}_b, \mathbf{V}_b$ is fetched from potentially non-contiguous GPU memory via the page table.

3.3.4 Impact

PagedAttention enables vLLM to achieve 2–4× higher throughput than HuggingFace Transformers and 1.5–2× over FasterTransformer, with near-zero memory waste. The paged design also enables efficient **prefix caching** (sharing KV-cache pages for common prompt prefixes across requests).

3.4 Sliding Window Attention

3.4.1 Definition

Each token attends only to the W most recent tokens:

$$\text{Attention}_{\text{SW}}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) = \frac{\sum_{j=\max(0,i-W+1)}^{i} e^{\mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d}} \mathbf{v}_j}{\sum_{j=\max(0,i-W+1)}^{i} e^{\mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d}}} \tag{3.6}$$

The attention matrix is banded, reducing memory from $O(n^2)$ to $O(nW)$ and enabling theoretically unbounded context length (through stacked sliding windows acting as a receptive field pyramid across layers).

3.4.2 Multi-Scale Sliding Window (Longformer, Beltagy et al., 2020)

Longformer uses two attention patterns:

- **Sliding window** of size W for local context (e.g., $W = 512$).
- **Global attention** on k pre-selected tokens (CLS, SEP, task-specific) that attend to the entire sequence.

Complexity: $O(nW + nk)$, capturing both local and global dependencies.

3.4.3 Mistral's Sliding Window

Mistral 7B uses sliding window attention with $W = 4096$, combined with a rolling KV-cache buffer (only the last W tokens are cached). The receptive field grows linearly with layer depth ℓ: a token at layer ℓ can attend to information from up to $\ell \cdot W$ tokens away through the layer stack.

3.5 Ring Attention (Liu et al., 2023)

3.5.1 Motivation

For extreme sequence lengths (>100K tokens), even Flash Attention's memory savings are insufficient for a single device. Ring Attention distributes the sequence across multiple devices in a ring topology.

3.5.2 Algorithm

Devices are arranged in a logical ring. Each device holds a block of $\mathbf{Q}$, and $\mathbf{K}, \mathbf{V}$ blocks circulate:

1. Each device computes local attention with its current KV block.
2. KV blocks are transmitted to the next device in the ring (via send/recv).
3. Repeat until all KV blocks have visited all devices.

Communication is overlapped with computation: while computing attention for block i, block $i + 1$ is being received. Total communication volume: $O(L \cdot d)$ per layer (linear in sequence length).

3.5.3 Striped Attention (Brandon et al., 2023)

A generalization of ring attention that uses a 2D device grid with block-cyclic distribution, reducing per-device memory further.

3.6 Other Efficient Attention Variants

3.6.1 Linear (Kernelized) Attention

Replace $\text{softmax}(\mathbf{QK}^\top)$ with $\phi(\mathbf{Q})\phi(\mathbf{K})^\top$ for a feature map ϕ, enabling the reordering $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top\mathbf{V})$ with complexity $O(n \cdot d^2)$:

$$\text{LinAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \phi(\mathbf{Q})(\phi(\mathbf{K})^\top\mathbf{V}) \tag{3.7}$$

Feature map choices:

- **Linear Transformer (Katharopoulos et al., 2020):** $\phi(x) = \text{ELU}(x) + 1$. Ensures positivity for the kernel approximation.
- **Performer (Choromanski et al., 2020):** $\phi(x) = \frac{1}{\sqrt{m}}(f(\omega_1^\top x), \ldots, f_m(\omega_m^\top x))$ using random Fourier features to approximate the Gaussian (RBF) kernel. $\omega_i \sim \mathcal{N}(0, \mathbf{I})$, $f = [\sin, \cos]$ or positive random features.
- **CosFormer:** $\phi(x) = \text{ReLU}(x)$ with a cosine-based positional re-weighting.
- **ABC (Attention with Bounded Causal):** Uses $\phi(x) = \exp(x)^\top$ with a bounded exponential for numerical stability.

Limitation: Linear attention struggles with tasks requiring precise token-level comparisons (the softmax's sharp selectivity is lost). Modern practice prefers Flash Attention (which preserves softmax quality) over linear attention for most applications.

3.6.2 Block Sparse Attention

Divide the attention matrix into fixed-size blocks and compute only a subset based on a pre-defined sparsity pattern:

- **Local + Strided (Child et al., 2019):** Attend to nearby tokens AND every k-th token.
- **Block-local:** Each token attends to all tokens within its spatial block.
- **Adaptive sparsity (Reformer):** Use LSH (locality-sensitive hashing) to group similar queries and keys, then compute attention only within groups.

3.6.3 Dilated / Strided Attention (Sparse Transformer)

Tokens attend to every k-th previous token (dilated pattern) or to fixed strides. This creates a sparsity pattern with $O(n\sqrt{n})$ complexity while maintaining long-range connectivity through the dilation factor.

3.6.4 Cross-Layer Attention Sharing

Reuse attention outputs from layer ℓ in layer $\ell + k$ without recomputing attention. The FFN still processes each layer independently. Reduces computation at the cost of slight quality degradation. Used as an optional optimization in some inference engines.

3.6.5 Infini-Attention (Google, 2024)

Combines local sliding window attention with a **compressive memory**: a linear attention module that maintains a fixed-size memory state across the entire sequence, effectively giving the model "infinite" context through a compressed representation of all past tokens.

$$\mathbf{M}_t = \mathbf{M}_{t-1} + \sigma(\mathbf{K}_t)^\top \mathbf{V}_t, \quad \mathbf{o}_{\text{mem},t} = \sigma(\mathbf{Q}_t)\mathbf{M}_t \tag{3.8}$$

The final output combines local attention and compressive memory through a learned gating mechanism.

3.6.6 Landmark Attention

Select a subset of "landmark" tokens that serve as global context summarizers. Every token attends to landmarks + local neighbors. The landmarks themselves attend globally. Complexity: $O(nk + nW)$ where k is the number of landmarks.

3.7 Attention Optimization Summary

Table 3.1: Comparison of attention mechanisms

Method	**Complexity**	**Quality vs MHA**	**Memory**	**Implementation**
Standard MHA	$O(n^2 d)$	100%	High	Universal
Flash Attention v2	$O(n^2 d)$	100%	Low (SRAM)	CUDA/Triton
Paged Attention	$O(n^2 d)$	100%	Optimal	vLLM
Sliding Window	$O(nWd)$	97–99%	Very Low	Mistral
Ring Attention	$O(n^2 d/P)$	100%	Distributed	Multi-GPU
Linear Attention	$O(nd^2)$	90–95%	Low	Custom
Block Sparse	$O(n\sqrt{n}d)$	93–97%	Low	Custom

Chapter 4

State Space Models and Mamba

State Space Models (SSMs) offer an alternative to attention with linear complexity in sequence length. They have deep roots in control theory and signal processing, recently adapted for language modeling.

4.1 Continuous-Time SSMs

4.1.1 State Space Representation

A linear time-invariant (LTI) SSM maps a 1-D input signal $u(t)$ to output $y(t)$ through a latent state $\mathbf{x}(t) \in \mathbb{R}^N$:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}u(t) \tag{4.1}$$

$$y(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}u(t) \tag{4.2}$$

where $\mathbf{A} \in \mathbb{R}^{N\times N}$ (state matrix), $\mathbf{B} \in \mathbb{R}^{N\times 1}$ (input vector), $\mathbf{C} \in \mathbb{R}^{1\times N}$ (output vector), $\mathbf{D} \in \mathbb{R}$ (skip connection, often omitted).

4.1.2 Discretization

To process discrete sequences, the continuous SSM is discretized with step size Δ. Using the **zero-order hold (ZOH)** method:

$$\overline{\mathbf{A}} = e^{\Delta\mathbf{A}} \tag{4.3}$$

$$\overline{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(e^{\Delta\mathbf{A}} - \mathbf{I})\Delta\mathbf{B} \approx \Delta\mathbf{B} \quad \text{(Euler approximation for small } \Delta) \tag{4.4}$$

The discrete recurrence becomes:

$$\mathbf{x}_k = \overline{\mathbf{A}}\mathbf{x}_{k-1} + \overline{\mathbf{B}}u_k \tag{4.5}$$

$$y_k = \mathbf{C}\mathbf{x}_k \tag{4.6}$$

4.1.3 Bilinear (Tustin) Discretization

An alternative with better numerical properties for some $\mathbf{A}$:

$$\overline{\mathbf{A}} = \left(\mathbf{I} - \frac{\Delta}{2}\mathbf{A}\right)^{-1}\left(\mathbf{I} + \frac{\Delta}{2}\mathbf{A}\right), \quad \overline{\mathbf{B}} = \left(\mathbf{I} - \frac{\Delta}{2}\mathbf{A}\right)^{-1}\Delta\mathbf{B} \tag{4.7}$$

4.2 S4: Structured State Space Sequence Model (Gu et al., 2022)

4.2.1 The HiPPO Matrix

Gu et al. (2022) introduced the HiPPO (High-order Polynomial Projection Operator) framework for initializing $\mathbf{A}$ to optimally compress input history. The HiPPO-LegS matrix encodes the Legendre polynomial basis:

$$\mathbf{A}_{nk} = -\begin{cases} \sqrt{(2n+1)(2k+1)} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases} \tag{4.8}$$

This ensures that the state $\mathbf{x}(t)$ captures the optimal polynomial approximation of the input history $u(\tau)$ for $\tau \leq t$. Intuitively, the HiPPO matrix creates a system that "remembers" past inputs in the Legendre basis.

4.2.2 Normal Plus Low-Rank (NPLR) Decomposition

S4's key computational insight: the HiPPO matrix can be decomposed as:

$$\mathbf{A} = \mathbf{\Lambda} - \mathbf{P}\mathbf{Q}^* \tag{4.9}$$

where $\mathbf{\Lambda}$ is diagonal and $\mathbf{P}\mathbf{Q}^*$ is low-rank. This decomposition, combined with Woodbury's identity, reduces the convolution computation from $O(N^2 L)$ to $O(NL)$ (with FFT-based acceleration: $O(L \log L)$).

4.2.3 Convolutional View

For a fixed Δ (or learned set of Δ), the SSM defines a *convolution kernel*:

$$\overline{\mathbf{K}} = \left(\mathbf{C}\overline{\mathbf{B}}, \mathbf{C}\overline{\mathbf{A}\mathbf{B}}, \ldots, \mathbf{C}\overline{\mathbf{A}}^{L-1}\overline{\mathbf{B}}\right) \tag{4.10}$$

During training, the SSM computes: $\mathbf{y} = \mathbf{u} * \overline{\mathbf{K}}$ (1D convolution, fully parallelizable). During inference (autoregressive), the recurrent form (Eq. 4.5) is used, requiring only $O(N^2)$ state update per token.

4.3 S5 and S6: Towards Mamba

4.3.1 S5: Multi-Input Multi-Output SSM (Smith et al., 2023)

S5 extends S4 from single-input/single-output to MIMO:

$$\mathbf{x}_k = \overline{\mathbf{A}}\mathbf{x}_{k-1} + \overline{\mathbf{B}}\mathbf{u}_k \tag{4.11}$$

$$\mathbf{y}_k = \mathbf{C}\mathbf{x}_k \tag{4.12}$$

where now $\mathbf{u}_k \in \mathbb{R}^H$, $\mathbf{y}_k \in \mathbb{R}^H$, $\mathbf{x}_k \in \mathbb{R}^{P \times H}$, $\overline{\mathbf{B}} \in \mathbb{R}^{P \times H}$, $\mathbf{C} \in \mathbb{R}^{H \times P}$. S5 uses **parallel scans** (Blelloch scan) for efficient parallel recurrence computation, achieving $O(L \log L)$ training time with $O(H)$ memory per token at inference.

4.3.2 S6 / Mamba: Selective SSM (Gu & Dao, 2023)

The critical limitation of S4/S5: $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \Delta)$ are **time-invariant**—they do not depend on input content. Mamba introduces **selectivity**:

$$\mathbf{B}_k = \text{Linear}_B(\mathbf{u}_k) \tag{4.13}$$

$$\mathbf{C}_k = \text{Linear}_C(\mathbf{u}_k) \tag{4.14}$$

$$\Delta_k = \text{softplus}(\text{Linear}_\Delta(\mathbf{u}_k)) \tag{4.15}$$

This allows the model to *selectively* propagate or forget information based on content—analogous to gating mechanisms in LSTMs or attention scores in transformers.

Why is selectivity hard? The convolutional view requires a fixed kernel $\overline{\mathbf{K}}$, impossible with input-dependent $\mathbf{B}_k, \mathbf{C}_k, \Delta_k$. Mamba solves this with a **hardware-aware parallel scan algorithm**: instead of materializing the full SSM, the scan computes prefix sums with dynamic $\overline{\mathbf{A}}_k = \exp(\Delta_k \mathbf{A})$ at each step, implemented via efficient CUDA kernels.

4.3.3 Mamba Block Structure

1. Input $\mathbf{x}$ is projected through a linear layer, then through a 1D depthwise convolution followed by SiLU (mixing short-range context).
2. The SSM (with selective parameters) processes the convolved signal.
3. A residual connection and output projection complete the block.

Mamba interleaves these blocks like transformer blocks but with $O(L)$ complexity.

Listing 4.1: Simplified Mamba block in PyTorch

```
class MambaBlock(nn.Module):
    def __init__(self, d_model, d_state=16, d_conv=4, expand=2):
        super().__init__()
        d_inner = d_model * expand
        self.in_proj = nn.Linear(d_model, d_inner * 2)
        self.conv1d = nn.Conv1d(d_inner, d_inner, d_conv,
            groups=d_inner, padding=d_conv-1)
        self.x_proj = nn.Linear(d_inner, d_state * 2 + 1)  # B, C,
            dt
        self.dt_proj = nn.Linear(d_state, d_inner)
        self.out_proj = nn.Linear(d_inner, d_model)

    def forward(self, x):
        # x: [batch, seqlen, d_model]
        x_and_res = self.in_proj(x)
        x, res = x_and_res.chunk(2, dim=-1)
        x = self.conv1d(x.transpose(1, 2))[:, :, :x.shape[1]]
        x = F.silu(x).transpose(1, 2)
        # Selective scan (simplified)
        B, C, dt = self.x_proj(x).split([d_state, d_state, 1], dim
            =-1)
        dt = F.softplus(dt)
        y = selective_scan(x, dt, B, C)  # custom CUDA kernel
        y = y * F.silu(res)
        return self.out_proj(y)
```

4.4 Mamba-2: Structured State Space Duality (Dao & Gu, 2024)

4.4.1 SSM-Attention Duality

Dao & Gu (2024) proved that SSMs and linear attention are two views of the same computation, connected through **structured (semi-separable) matrices**. An SSM with state size N is equivalent to a semiseparable matrix of rank N:

$$\mathbf{M}_{ij} = \mathbf{C}_i \left(\prod_{k=j+1}^{i} \overline{\mathbf{A}}_k \right) \overline{\mathbf{B}}_j \tag{4.16}$$

This can be framed as either state-space recurrence OR block-decomposed matrix multiplication—the latter being more amenable to tensor core acceleration.

4.4.2 Key Improvements over Mamba-1

- **Tensor core optimization:** Reformulates the SSM as a block-decomposed matrix multiply, achieving 2–8× speedup on modern GPUs.
- **Scalar-times-identity A:** Simplifies the state to $\mathbf{A} = a \cdot \mathbf{I}$, making the SSM equivalent to a weighted cumulative sum for each channel independently.
- **Larger state dimension:** Efficiency gains allow increasing N from 16 to 64–256, expanding the model's effective "memory" capacity.
- **Hybrid architectures:** Mamba-2 blocks can freely interleave with attention blocks, combining strengths: attention for precise token comparisons, SSM for long-range context compression.

4.5 RWKV: Receptance Weighted Key Value (Peng et al., 2023)

RWKV blends RNN and transformer properties. Its core time-mixing mechanism:

$$\mathbf{r}_t = \mathbf{W}_r \cdot (\mu_r \odot \mathbf{x}_t + (1 - \mu_r) \odot \mathbf{x}_{t-1}) \tag{4.17}$$

$$\mathbf{k}_t = \mathbf{W}_k \cdot (\mu_k \odot \mathbf{x}_t + (1 - \mu_k) \odot \mathbf{x}_{t-1}) \tag{4.18}$$

$$\mathbf{v}_t = \mathbf{W}_v \cdot (\mu_v \odot \mathbf{x}_t + (1 - \mu_v) \odot \mathbf{x}_{t-1}) \tag{4.19}$$

$$\mathbf{wkv}_t = \frac{\sum_{i=1}^{t-1} e^{-(t-1-i)w+\mathbf{k}_i} \odot \mathbf{v}_i + e^{u+\mathbf{k}_t} \odot \mathbf{v}_t}{\sum_{i=1}^{t-1} e^{-(t-1-i)w+\mathbf{k}_i} + e^{u+\mathbf{k}_t}} \tag{4.20}$$

$$\mathbf{o}_t = \mathbf{W}_o \cdot (\sigma(\mathbf{r}_t) \odot \mathbf{wkv}_t) \tag{4.21}$$

where w (decay) and u (first-token bonus) are learned per-channel parameters. The μ parameters interpolate between current and previous token for each of R, K, V.

RWKV v5/v6 (Eagle/Finch): Introduces data-dependent time decay, multi-headed WKV, and dynamic token mixing ratios for improved performance.

4.6 Other SSM Architectures

4.6.1 H3: Hungry Hungry Hippos (Fu et al., 2023)

Demonstrated that stacking two SSMs with multiplicative interactions emulates attention:

$$\mathbf{o} = \mathrm{SSM}_{\mathrm{shift}}(\mathbf{Q} \odot \mathrm{SSM}(\mathbf{K} \odot \mathbf{V})) \tag{4.22}$$

One SSM processes keys, another processes values; their element-wise product creates a quadratic interaction approximating attention without the $O(n^2)$ cost.

4.6.2 Hyena (Poli et al., 2023)

Replaces attention with a long convolution parameterized by an MLP:

$$\mathbf{K}[t] = \mathrm{MLP}_{\mathrm{proj}}\big(\mathrm{MLP}_{\mathrm{body}}(t/L)\big) \tag{4.23}$$

The learned kernel is applied via FFT ($O(L \log L)$). HyenaHierarchy stacks multiple Hyena operators with different filter lengths for multi-scale processing.

4.6.3 Griffin (De et al., 2024)

Introduces **RG-LRU (Real-Gated Linear Recurrent Unit)**:

$$\mathbf{h}_t = \mathbf{a}_t \odot \mathbf{h}_{t-1} + \sqrt{1 - \mathbf{a}_t^2} \odot (\mathbf{b}_t \mathbf{x}_t) \tag{4.24}$$

where $\mathbf{a}_t = \sigma(\mathbf{W}_a \mathbf{x}_t)$ is a data-dependent gating factor. Griffin matches Mamba's quality while being simpler to implement and train.

4.6.4 Jamba (AI21 Labs, 2024)

A hybrid architecture interleaving Mamba SSM blocks with transformer attention blocks AND MoE layers. Combines SSM efficiency, attention precision, and MoE capacity scaling in a single model.

4.7 Comparing SSM Architectures

Table 4.1: Comparison of SSM-based architectures

Model	Complexity	Selectivity	Throughput	PPL vs Transformer
S4	$O(L \log L)$	No	Moderate	Worse
S5	$O(L \log L)$	No	Moderate	Comparable (small)
Mamba	$O(L)$	Yes	Very High	Comparable
Mamba-2	$O(L)$	Yes	Highest	Comparable or Better
RWKV-v6	$O(L)$	Yes	Very High	Comparable
Griffin	$O(L)$	Yes	High	Comparable
Jamba (hybrid)	$O(L)$	Partial	High	Comparable

4.8 Mathematical Intuition: Why SSMs Work for Language

SSMs encode history through the state $\mathbf{x}(t)$, which acts as a **compressed memory**. The HiPPO theory shows that the state optimally encodes a Legendre polynomial approximation of the input signal. For language, this means:

- Low-order polynomials capture long-range, slowly-varying semantic context.
- Higher-order components capture rapidly-changing local syntax and token identities.
- Selectivity (Mamba) lets the model decide *what* to store in this compressed memory based on content, overcoming the fixed-encoding limitation.

This perspective unifies SSMs with other linear-complexity approaches: RWKV's exponential decay, Griffin's gated recurrence, and Mamba's selective SSM are all ways to implement *content-aware compression* of the input history.

Chapter 5

Quantization: Theory and Practice

Quantization reduces the bit-width of model weights (and optionally activations), shrinking memory footprint and accelerating inference. This chapter covers the full taxonomy from fundamentals to cutting-edge methods.

5.1 Quantization Fundamentals

5.1.1 Uniform Affine Quantization

The most common scheme maps FP32 values to an integer range $[0, 2^b - 1]$:

$$r = s \cdot (q - z) \quad \text{(dequantization)} \tag{5.1}$$

$$q = \text{round}\left(\frac{r}{s}\right) + z, \quad \text{clamped to } [0, 2^b - 1] \tag{5.2}$$

where r is the real value, q the quantized integer, s the scale, z the zero-point. For **symmetric** quantization, $z = 0$, range $[-2^{b-1}, 2^{b-1} - 1]$. Symmetric is more common for LLM weights (simpler, natural zero alignment).

5.1.2 Scale Determination Strategies

- **MinMax:** $s = \frac{\max(|r_{\min}|, |r_{\max}|)}{2^{b-1}-1}$. Simple, but sensitive to outliers.
- **Percentile (99.9%):** Clip extreme outliers, then apply MinMax. Reduces quantization error at minor clipping loss.
- **MSE:** Minimize $\mathbb{E}[(r - s \cdot \text{round}(r/s))^2]$ over the calibration dataset. Solved via grid search or by setting $s = \frac{\mathbb{E}[r \cdot q]}{\mathbb{E}[q^2]}$ (analytical solution for fixed $z = 0$).
- **Cross-entropy:** Minimize output distribution divergence (not weight error directly). More accurate but computationally expensive, requiring forward passes.

5.1.3 Quantization Granularity

- **Per-tensor:** One scale for the entire weight tensor. No overhead, worst accuracy.
- **Per-channel (row-wise):** One scale per output channel (row). Handles varying magnitudes across channels.
- **Per-group:** Divide channels into groups of g (typically 64 or 128). One scale per group. Best accuracy-size tradeoff.

- **Per-token (for activations):** One scale per token in the batch. Used in conjunction with per-channel weight quantization.

5.1.4 Quantization-Aware Training (QAT)

Simulates quantization during training via **fake quantization**:

$$y = s \cdot \text{round}(\text{clamp}(x/s + z, 0, 2^b - 1)) - z \cdot s \tag{5.3}$$

The backward pass uses the **Straight-Through Estimator (STE)**:

$$\frac{\partial y}{\partial x} = \mathbf{1}_{[0,2^b-1]}(q) \quad \text{(gradient passes through rounding unchanged)} \tag{5.4}$$

5.1.5 Post-Training Quantization (PTQ)

No training/fine-tuning required. Uses a small calibration dataset (128–1024 samples) to determine scales. Main challenge: **outlier channels**—weight channels with 10–100× larger magnitude than typical channels dominate quantization error.

5.2 LLM.int8() — Mixed-Precision Quantization (Dettmers et al., 2022)

5.2.1 The Outlier Problem

Dettmers et al. discovered that LLMs > 6.7B parameters exhibit **emergent outliers**: specific feature dimensions in specific layers have values 20–100× larger than others. These outliers constitute $< 0.1\%$ of values but are concentrated in specific columns. They break uniform quantization.

5.2.2 LLM.int8() Algorithm

1. **Outlier detection:** Compute activation magnitude per hidden dimension. Mark columns where any absolute value exceeds threshold (typically 6.0) as outliers.
2. **Mixed-precision matmul** for $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top$:
   - Outlier columns of $\mathbf{X}$ and corresponding rows of $\mathbf{W}$: kept in FP16.
   - All other values: quantized to INT8.
   - INT8 and FP16 matmuls run separately, results summed.
3. **Per-channel quantization:** INT8 weights use row-wise quantization; INT8 activations use column-wise.

Performance: Zero degradation vs FP16. Memory reduction: ∼2× (FP16→INT8 for 99.9% of parameters). Standard approach for 8-bit inference.

5.3 GPTQ: Accurate Post-Training Quantization (Frantar et al., 2023)

5.3.1 Optimal Brain Quantization (OBQ) Foundation

OBQ formulates layer-wise quantization as minimizing output error:

$$\min_{\widehat{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \widehat{\mathbf{W}}\mathbf{X}\|_F^2 = \min_{\Delta\mathbf{W}} \mathrm{Tr}\left(\Delta\mathbf{W}^\top \Delta\mathbf{W} \cdot \mathbf{X}\mathbf{X}^\top\right) \tag{5.5}$$

For a single weight w_p in a row $\mathbf{w}$, the optimal update to compensate for quantizing w_p is:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{w_p - \widehat{w}_p}{[\mathbf{H}^{-1}]_{pp}} \cdot \mathbf{H}^{-1}_{:,p} \tag{5.6}$$

where $\mathbf{H}^{-1} = (\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I})^{-1}$ is the inverse Hessian. This *propagates* the quantization error to all remaining weights to maintain output fidelity.

5.3.2 The OBQ Iterative Algorithm

1. Compute inverse Hessian $\mathbf{H}^{-1}$ from calibration data.
2. For each column p (ordered by importance):
   (a) Quantize $w_p \to \widehat{w}_p$ (round to grid).
   (b) Compensate all remaining unquantized weights using the formula above.

5.3.3 GPTQ Optimizations

OBQ is $O(d^3)$ per row, impractical for $d = 4096+$ LLM layers. GPTQ introduces:

- **Lazy batch updates:** Accumulate compensation updates and apply them in batches of 128 columns, amortizing memory access.
- **Cholesky reformulation:** Pre-compute $\mathbf{H}^{-1} = \mathbf{L}\mathbf{L}^\top$ to avoid repeated inversions.
- **Column reordering:** Sort by descending Hessian diagonal—quantize "most determined" weights first.
- **Damping:** Add $\lambda\mathbf{I}$ to $\mathbf{H}$ proportional to the mean diagonal, preventing numerical instability.

Time: $O(d^2 \cdot B)$ per layer. GPTQ quantizes a 175B model in ∼4 GPU-hours.

5.3.4 GPTQ Practical Usage

Listing 5.1: GPTQ with AutoGPTQ

```
from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig
from transformers import AutoTokenizer

quantize_config = BaseQuantizeConfig(
    bits=4,                 # 4-bit quantization
    group_size=128,         # per-group granularity
    desc_act=True,          # act-order (descending Hessian)
    damp_percent=0.01,      # Hessian dampening
```

```
)
model = AutoGPTQForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    quantize_config=quantize_config
)
```

5.4 AWQ: Activation-Aware Weight Quantization (Lin et al., 2024)

5.4.1 Core Insight

Not all weight channels are equally important—channels with large activation magnitudes contribute disproportionately to the output. Quantizing them naively causes large errors.

5.4.2 Per-Channel Scaling Transformation

Instead of protecting salient weights from quantization, AWQ **scales** them:

$$\mathbf{W}'_{:,j} = \mathbf{W}_{:,j} \cdot \frac{1}{s_j}, \quad \mathbf{X}'_{:,j} = \mathbf{X}_{:,j} \cdot s_j \tag{5.7}$$

The matmul is invariant: $\mathbf{W}'\mathbf{X}'^{\top} = \mathbf{W}\mathbf{X}^{\top}$. Scaling factors are:

$$s_j = \frac{\max(|\mathbf{X}_{:,j}|)^{\alpha}}{\max(|\mathbf{W}_{:,j}|)^{1-\alpha}} \tag{5.8}$$

where $\alpha \in [0, 1]$ balances activation vs. weight magnitude. Grid-searched on calibration data (128 samples).

5.4.3 Algorithm

1. Collect activation statistics on calibration data.
2. Compute per-channel scales s_j via grid search minimizing layer-wise MSE.
3. Scale weights and fuse scaling into preceding normalization layer.
4. Apply uniform quantization (typically INT4, per-group granularity).

Advantages over GPTQ: 1.5–2× faster quantization; competitive accuracy; no runtime overhead (scaling fused into existing layers).

5.5 GGUF / GGML: CPU-Centric Quantization

5.5.1 The GGML Ecosystem

GGML is a C/C++ tensor library for CPU inference. GGUF (Georgi Gerganov Unified Format) is its file format successor. The `llama.cpp` project popularized this ecosystem for CPU, Apple Silicon (Metal), and edge device inference.

5.5.2 Quantization Types

Table 5.1: Common GGUF quantization types

Type	Bits (avg)	Block	Quality	Speed
Q4_0	4.5	32	Baseline	Fastest
Q4_1	4.5	32	+0.1 vs Q4_0	Fast
Q4_K_M	4.5	256	++	Medium
Q4_K_S	4.5	256	+	Fast
Q5_0	5.5	32	+	Fast
Q5_K_M	5.5	256	++	Medium
Q6_K	6.6	256	+++	Slower
Q8_0	8.5	32	++++	Slow (near-FP16)
IQ4_XS	4.25	256	++ with importance	Medium
IQ3_XXS	3.06	256	Acceptable for 70B+	Fast
IQ2_XXS	2.06	256	Use only for >70B models	Very fast

Block-wise quantization: For a block of B weights:

$$d = \max_i |w_i| \cdot \frac{1}{2^{b-1}-1} \quad \text{(block scale, stored in FP16)} \tag{5.9}$$

$$q_i = \text{round}\left(\frac{w_i}{d}\right), \quad \text{clamped to } \pm(2^{b-1}-1) \tag{5.10}$$

The "K" variants (by Kawrakow) use importance matrices computed from calibration data to weight quantization error by its contribution to the final loss. "IQ" types use importance-aware quantization with non-uniform grids.

Listing 5.2: GGUF quantization workflow

```
# Convert HF model to FP16 GGUF
python convert-hf-to-gguf.py /path/to/model --outtype f16

# Quantize (Q4_K_M recommended for best quality/size)
./quantize model-f16.gguf model-Q4_K_M.gguf Q4_K_M
```

5.6 NF4: NormalFloat 4-bit (Dettmers et al., 2023)

5.6.1 Information-Theoretic Quantization

NF4 uses a data-dependent quantization grid optimized for normally distributed weights:

$$q_i = \frac{1}{2}\left(Q\left(\frac{i}{16}\right) + Q\left(\frac{i+1}{16}\right)\right), \quad i = 0, \ldots, 15 \tag{5.11}$$

where $Q(p) = \Phi^{-1}(p)$ is the inverse CDF of $\mathcal{N}(0,1)$. This places more bins near zero (where weights concentrate), reducing expected quantization error by 20–30% vs. uniform INT4.

5.6.2 Block-wise Implementation

Weights are divided into blocks of 64 elements. Each block has one FP32 absolute-max scale. Scales are double-quantized to FP8. Dequantization is: $w = q \cdot s_{\text{block}} \cdot s_{\text{page}}$ for hierarchical quantization.

5.7 HQQ: Half-Quadratic Quantization (Badri & Shaji, 2023)

5.7.1 Formulation as Optimization Problem

HQQ frames quantization as minimizing:

$$\min_{\mathbf{W}_q} \|\mathbf{W} - \mathbf{W}_q\|_F^2 + \lambda \cdot \text{card}(\mathbf{W}_q) \tag{5.12}$$

The half-quadratic splitting alternates between:

1. **Quantization:** Project $\mathbf{W}$ to the quantization grid (standard rounding).
2. **Refinement:** Optimize scales via least squares with fixed quantized values.

HQQ v1 requires NO calibration data (zero-data PTQ). HQQ v2 uses calibration for better accuracy. HQQ+ extends to 1.5–2 bit via fine-tuning.

Listing 5.3: HQQ usage

```
from hqq.engine.hf import HQQModelForCausalLM
from hqq.core.quantize import BaseQuantizeConfig

quant_config = BaseQuantizeConfig(
    nbits=4, group_size=64, axis=1
)
model = HQQModelForCausalLM.quantize(model, quant_config)
```

5.8 QUIP and QuIP# (Chee et al., 2024)

5.8.1 Incoherence Processing

QUIP applies random orthogonal transforms to "whiten" the weight matrix:

$$\widetilde{\mathbf{W}} = \mathbf{U}\mathbf{W}\mathbf{V}^\top \tag{5.13}$$

where $\mathbf{U}, \mathbf{V}$ are random orthogonal matrices (Randomized Hadamard Transforms). $\widetilde{\mathbf{W}}$ has near-Gaussian entries, making uniform quantization near-optimal.

5.8.2 QuIP# Improvements

- **Lattice codebooks:** Uses the E_8 lattice (densest sphere packing in 8D):

$$\widehat{\mathbf{w}} = \underset{\mathbf{c} \in E_8}{\text{argmin}} \|\mathbf{w} - s \cdot \mathbf{c}\|^2 \tag{5.14}$$

- **Finetuning:** Adaptive rounding via 10–50 steps of MSE or cross-entropy loss.
- **Result:** 2-bit LLaMA-2 70B with <5% perplexity degradation, state-of-the-art for extreme compression.

5.9 SmoothQuant (Xiao et al., 2023)

5.9.1 Activation-Weight Migration

SmoothQuant migrates quantization difficulty from activations to weights:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^\top = (\mathbf{X} \cdot \mathrm{diag}(\mathbf{s})^{-1}) \cdot (\mathbf{W} \cdot \mathrm{diag}(\mathbf{s}))^\top \tag{5.15}$$

where $s_j = \max(|X_{:,j}|)^\alpha / \max(|W_{:,j}|)^{1-\alpha}$. The migration strength $\alpha \in [0, 1]$ balances the burden between activations and weights. Enables W8A8 (INT8 weights + INT8 activations) with negligible accuracy loss, delivering $\sim$2$\times$ throughput.

5.10 Additional Quantization Methods

5.10.1 AQLM (Additive Quantization, 2024)

Uses multi-codebook vector quantization:

$$\widehat{\mathbf{w}} = \sum_{m=1}^{M} \mathbf{c}_{i_m}^{(m)} \tag{5.16}$$

Multiple codebooks additively refine the reconstruction. Achieves 2–2.5 bits with strong performance through end-to-end codebook learning.

5.10.2 SpQR (Sparse-Quantized Representation, 2023)

Keeps a small fraction of weights in FP16 as "sparse outliers" while quantizing the rest to 3–4 bits. Achieves near-lossless compression for LLaMA.

5.10.3 FP8 / E4M3 / E5M2

- **E4M3:** 4-bit exponent, 3-bit mantissa. Range $[2^{-9}, 448]$. For training.
- **E5M2:** 5-bit exponent, 2-bit mantissa. Range $[2^{-22}, 57344]$. For inference.
- Native on H100 GPUs via Transformer Engine with $\sim$2$\times$ throughput.

5.10.4 MX-FP (Microscaling, 2024)

Industry standard: shared 8-bit exponent per 32 elements, individual E2M1 mantissas. Fine-grained dynamic range with high storage efficiency.

5.10.5 FlexRound (2024)

Learns per-weight rounding decisions (round vs floor vs ceil) via gradient-based optimization on a small calibration set, pushing quantization error to less sensitive weights.

5.10.6 Binary/Ternary Quantization

- **BinaryConnect/BNN:** Weights in $\{-1, +1\}$. Extreme compression but high quality loss.
- **Ternary (BiT):** Weights in $\{-w_0, 0, +w_0\}$ with learned scaling w_0.
- **BitNet b1.58:** Ternary weights $(-1, 0, 1)$ with per-channel scaling, matching FP16 quality at 1/16 memory for models trained with this scheme from scratch.

5.11 Comparison of PTQ Methods

Table 5.2: Comprehensive comparison of post-training quantization methods

Method	Bits	Calib.	7B Time	Key Innovation
LLM.int8()	8	128 samp.	∼1 min	Mixed-precision outlier handling
GPTQ	2–8	128 samp.	∼1 hr	Hessian-based error compensation
AWQ	4	128 samp.	∼20 min	Activation-aware scaling
HQQ v1	4	None	∼10 sec	Zero-data; half-quadratic splitting
QuIP#	2	128 samp.	∼2 hr	Lattice codebooks + incoherence
SmoothQuant	W8A8	128 samp.	∼10 min	Activation-weight smoothening
GGUF Q4_K	4.5	None	∼2 min	Block-wise with importance
GGUF IQ4_XS	4.25	Importance	∼5 min	Importance-aware grid
NF4	4	None	<1 sec	Data-dependent quantile grid
AQLM	2	Full data	∼10 hr	Additive vector quantization
SpQR	3–4	128 samp.	∼2 hr	Sparse outlier preservation
FlexRound	4	128 samp.	∼30 min	Learned rounding decisions

5.12 Practical Quantization Guide

For most local deployment scenarios:

- **CPU (llama.cpp):** Q4_K_M for balance; IQ4_XS for maximum quality; IQ2_XXS for extreme memory constraints on large (70B+) models.
- **GPU with vLLM:** AWQ or GPTQ INT4 with group_size=128.
- **Training/QLoRA:** NF4 with double quantization (bitsandbytes).
- **Maximum speed:** FP8 (E4M3) on H100; GPTQ INT4 with EXL2 kernel.
- **Edge devices:** GGUF Q4_0 or Q4_K_S with Metal/Vulkan backend.

Chapter 6

Pruning and Sparsity

Pruning removes weights or structures from the network, trading a small amount of accuracy for significant compute and memory savings.

6.1 Unstructured (Magnitude) Pruning

6.1.1 Definition

Set individual weights to zero based on magnitude. For a pruning ratio p, keep top $(1-p)$ fraction:

$$\mathbf{W}_{\text{pruned}} = \mathbf{W} \odot \mathbf{M}, \quad M_{ij} = \begin{cases} 1 & \text{if } |W_{ij}| \geq \tau \\ 0 & \text{otherwise} \end{cases} \tag{6.1}$$

where τ is the $(p \cdot 100)$-th percentile of $|W_{ij}|$ over the layer.

6.1.2 Sparsity Schedules

- **One-shot:** Prune once, fine-tune remaining weights.
- **Iterative:** Prune, fine-tune, prune again—typically 3–10 rounds. Allows network recovery between stages.
- **Gradual (cubic schedule):** $p_t = p_{\text{final}} \cdot (1 - (1 - t/T)^3)$, smoothly increasing sparsity.
- **Automated Gradual Pruning (AGP):** Adaptively adjusts pruning rate based on loss trajectory.

6.1.3 The Lottery Ticket Hypothesis

Frankle & Carbin (2019): within a randomly initialized dense network, there exists a sparse subnetwork ("winning ticket") that, when trained in isolation with its **original initialization**, matches the full network's performance. Key: use original initialization, not random re-initialization after pruning.

6.1.4 Movement Pruning (Sanh et al., 2020)

Learn importance scores $\mathbf{s}$ during fine-tuning that *move* toward zero for unimportant weights and away from zero for important ones. The decision to prune is based on $\mathbf{s}$, not

weight magnitude. Gradient of $\mathbf{s}$ is:

$$\frac{\partial \mathcal{L}}{\partial s_i} = \frac{\partial \mathcal{L}}{\partial w_i} \cdot \frac{\partial w_i}{\partial s_i} \tag{6.2}$$

where $w_i = s_i \odot \tilde{w}_i$ ($\tilde{w}_i$ is a score-independent base weight).

6.2 SparseGPT (Frantar & Alistarh, 2023)

6.2.1 Motivation

Standard magnitude pruning + fine-tuning is prohibitively expensive for LLMs. SparseGPT achieves **one-shot** pruning to 50–60% sparsity with negligible perplexity increase and no fine-tuning.

6.2.2 Algorithm: Extending OBQ to Pruning

1. Compute inverse Hessian $\mathbf{H}^{-1} = (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I})^{-1}$ from calibration data.
2. **Column-adaptive masking:** For each column j (Hessian-determined order):
   (a) Rank weights to prune by Hessian-normalized importance: $|W_{ij}|/\sqrt{[\mathbf{H}^{-1}]_{jj}}$.
   (b) Remove masked weights: $\mathbf{W} \leftarrow \mathbf{W} \odot \mathbf{M}$.
   (c) **Compensate** surviving weights: update to minimize local reconstruction error.
3. Uses batched column updates (like GPTQ) for efficiency.

The compensation step solves a local least-squares problem, adjusting surviving weights so the pruned layer approximates the original behavior. This is what distinguishes SparseGPT from naive magnitude pruning.

6.2.3 Performance

At 50% unstructured sparsity: $< 1\%$ perplexity increase on LLaMA-2. At 60%: $< 2\%$. 50% sparsity + 4-bit quantization: $\sim 8\times$ compression in a single pass.

6.3 Wanda: Pruning by Weights AND Activations (Sun et al., 2024)

6.3.1 Core Idea

Weight magnitude alone is a poor criterion—importance also depends on activation magnitude:

$$\mathbf{I}_{ij} = |\mathbf{W}_{ij}| \cdot \|\mathbf{X}_{:,j}\|_2 \tag{6.3}$$

where $\|\mathbf{X}_{:,j}\|_2$ is the L2 norm of the j-th input activation across the calibration set.

6.3.2 Comparison with SparseGPT

- **Wanda:** $O(d^2)$ per layer. No matrix inversion. Purely heuristic. 100–1000× faster than SparseGPT. Excellent at moderate sparsity (50%).
- **SparseGPT:** $O(d^2 B)$ per layer. Least-squares optimal compensation. Better at extreme sparsity ($> 60\%$).

6.4 Structured Pruning

6.4.1 Motivation

Unstructured sparsity is hard to accelerate on GPUs (needs specific patterns like 2:4). Structured pruning removes entire columns, rows, or heads, producing smaller dense matrices that benefit from conventional hardware.

6.4.2 Types

- **Width pruning:** Remove entire columns/rows by deleting input/output features.
- **Head pruning:** Remove entire attention heads with least contribution to task performance.
- **Layer/depth pruning:** Remove entire transformer layers. LLMs can be more aggressively depth-pruned than width-pruned.
- **Block pruning:** Remove contiguous blocks of weights.
- **N:M sparsity:** At most N non-zero in each contiguous block of M. NVIDIA's 2:4 doubles matmul throughput on Ampere+ via Sparse Tensor Cores.

6.4.3 Importance Estimation for Structured Pruning

- **Magnitude:** Sum/mean of absolute weight values in the structure.
- **Taylor (gradient-based):** $I(\theta) \approx (\theta \cdot \nabla_\theta \mathcal{L})^2$—first-order approximation of loss change.
- **SNIP (Single-shot Network Pruning):** Connection sensitivity at initialization: $I(\theta) = |\theta \cdot \nabla_\theta \mathcal{L}|$.
- **GraSP (Gradient Signal Preservation):** $I(\theta) = -\theta \odot \mathbf{H}\theta$—preserves gradient flow.
- **LLM-Pruner:** Dependency-based structured pruning of LLMs using only calibration data (no gradients). Estimates dependency graphs between layers.

6.5 N:M Sparsity and Hardware

6.5.1 NVIDIA 2:4 Pattern

Ampere Sparse Tensor Cores: each contiguous block of 4 elements has at most 2 non-zeros. Doubles matmul throughput. Obtained via:

1. Train dense model.
2. Prune each block-of-4 to 2 largest magnitudes.
3. Fine-tune non-zeros for 5–10% of original steps.

Listing 6.1: 2:4 sparsity in PyTorch

```
from torch.sparse import to_sparse_semi_structured
weight_24 = to_sparse_semi_structured(weight_pruned)  # 2:4
output = torch.nn.functional.linear(x, weight_24)
# Hardware-accelerated on Ampere+ GPUs
```

Libraries: `torch.sparse`, NVIDIA ASP (Automatic Sparsity recipe), Neural Magic `sparsify`.

6.6 DSnoT: Dynamic Sparse No-Training Pruning (2024)

Achieves pruning ratios up to 80% with zero training/calibration by analyzing weight similarity patterns. Clusters similar weights and keeps cluster representatives, exploiting natural redundancy in over-parameterized layers.

6.7 SliceGPT (Ashkboos et al., 2024)

Uses PCA on $\mathbf{W}\mathbf{W}^\top$ to find orthogonal transformation $\mathbf{Q}$ that concentrates information in first k columns of $\mathbf{Q}^\top\mathbf{W}\mathbf{Q}$, then drops the remaining $(d-k)$ columns. Reduces matrix dimensions via rotational sparsity.

6.8 Pruning + Quantization Synergy

Modern approaches combine both:

- **SparseGPT:** Simultaneous pruning + quantization in one pass.
- **SpQR:** Sparse outlier preservation + quantization.
- **Optimal combination:** 50% sparsity + 4-bit = cumulative $\sim$8$\times$ memory reduction with $<$2% quality loss.

Chapter 7

Knowledge Distillation

Knowledge distillation transfers "dark knowledge" from a large teacher model to a smaller student, enabling the student to achieve better performance than training from scratch.

7.1 Foundational Distillation (Hinton et al., 2015)

7.1.1 Soft Targets and Temperature

Train the student on the teacher's **softened probability distribution**:

$$p_i^\tau = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}} \tag{7.1}$$

where $\tau > 1$ softens the distribution. The distillation loss:

$$\mathcal{L}_{\text{KD}} = \alpha \cdot \tau^2 \cdot \text{KL}(p_{\text{student}}^\tau \| p_{\text{teacher}}^\tau) + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}(y_{\text{student}}, y_{\text{true}}) \tag{7.2}$$

The τ^2 compensates for gradient magnitude reduction by temperature. α trades off between mimicking the teacher and fitting ground truth.

7.1.2 Why Temperature Helps

At $\tau = 1$, teacher output is near-deterministic (e.g., 0.99 for top class), revealing almost no information about class relationships. At intermediate τ (2–20), the distribution reveals which classes the teacher considers plausible—"dark knowledge" that helps the student generalize.

7.2 Logit-Based Distillation Variants

7.2.1 Standard KD (Soft Targets)

The KL divergence formulation above. The student learns the teacher's relative confidence between classes.

7.2.2 DistilBERT (Sanh et al., 2019)

- **Triple loss:** $\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{KD}} + \mathcal{L}_{\text{COS}}$ (cosine distance between hidden states).
- **Initialization:** Student initialized from every other layer of teacher.
- **Result:** 40% parameter reduction, 97% of BERT performance.

7.2.3 TinyBERT (Jiao et al., 2020)

Two-stage distillation:

1. **General distillation:** Pre-train student with teacher on large-scale corpus.
2. **Task-specific distillation:** Fine-tune student with data augmentation.

7.2.4 MiniLLM (Gu et al., 2024)

Distills LLMs by minimizing **reverse KLD**: $\text{KL}(p_{\text{student}} \| p_{\text{teacher}})$ instead of forward KL. Encourages mode-seeking behavior—the student focuses on the teacher's most confident predictions rather than covering the full distribution.

7.3 Feature-Based Distillation

7.3.1 Hidden State Matching

Match intermediate representations:

$$\mathcal{L}_{\text{feat}} = \sum_{\ell \in \mathcal{S}} \| \mathbf{H}_s^{(\ell)} \mathbf{W}_\ell - \mathbf{H}_t^{(g(\ell))} \|_F^2 \tag{7.3}$$

where $\mathcal{S}$ is the set of student layers, $g(\ell)$ maps to the corresponding teacher layer, and $\mathbf{W}_\ell$ aligns dimensions.

7.3.2 Attention Transfer (Zagoruyko & Komodakis, 2017)

Transfer teacher's attention maps:

$$\mathcal{L}_{\text{AT}} = \sum_{\ell} \left\| \frac{\mathbf{A}_s^{(\ell)}}{\|\mathbf{A}_s^{(\ell)}\|_2} - \frac{\mathbf{A}_t^{(\ell)}}{\|\mathbf{A}_t^{(\ell)}\|_2} \right\|_p \tag{7.4}$$

where $\mathbf{A}^{(\ell)}$ is the spatial attention map (channel-pooled feature correlations or attention weights).

7.3.3 Relation-Based Distillation

Transfer **relationships** between samples rather than per-sample features. For a mini-batch:

$$\mathcal{L}_{\text{RKD}} = \sum_{(i,j)} \| \psi(\mathbf{h}_s^{(i)}, \mathbf{h}_s^{(j)}) - \psi(\mathbf{h}_t^{(i)}, \mathbf{h}_t^{(j)}) \|^2 \tag{7.5}$$

where ψ is Euclidean distance, cosine similarity, or angular difference. Captures the *structure* of the teacher's feature space.

7.3.4 FitNet (Romero et al., 2015)

Uses "hints" from the teacher's middle layers to guide the student's intermediate representations. The student learns a "guided" layer that regresses to the teacher's hint layer.

7.4 Multi-Teacher and Ensemble Distillation

7.4.1 Multi-Teacher Distillation

A single student learns from multiple teachers simultaneously:

$$\mathcal{L}_{\text{MT}} = \sum_{k=1}^{K} \lambda_k \cdot \mathcal{L}_{\text{KD}}^{(k)}(p_s^\tau \| p_{t_k}^\tau) \tag{7.6}$$

Weights λ_k can be based on teacher performance. The student learns an implicit ensemble.

7.4.2 Ensemble Distillation (Born-Again Networks)

Train a student to mimic the averaged predictions of an ensemble. Special case: **self-distillation**—a student with identical architecture as the teacher often outperforms the teacher through ensemble-like effects (diversity from different random seeds during distillation).

7.5 Data-Free Distillation

When original training data is unavailable (proprietary teachers):

- **Inversion:** Optimize synthetic inputs: $\mathbf{x}^* = \text{argmax}_{\mathbf{x}} \text{score}_t(\mathbf{x}, c)$ to maximize teacher confidence in class c.
- **Noise-based:** Feed random noise to teacher and use its outputs as pseudo-labels. Surprisingly effective with augmentation.
- **ZS-Data:** Use unlabeled public data as proxy for the teacher's training distribution.
- **DeepInversion:** Optimize both images AND class-conditional batch normalization statistics to synthesize realistic training data.

7.6 LLM-Specific Distillation Methods

7.6.1 Sequence-Level KD (SeqKD)

For autoregressive LMs, distill at the sequence level:

$$\mathcal{L}_{\text{SeqKD}} = -\sum_{t=1}^{T} \sum_{v=1}^{V} p_t^{(t)}(v|x_{<t}) \cdot \log p_s(v|x_{<t}) \tag{7.7}$$

Standard teacher-forcing cross-entropy applied to the teacher's distribution.

7.6.2 Chat Distillation (Orca / Orca-2, 2023–2024)

Microsoft's approach:

1. Use GPT-4 to generate detailed **step-by-step explanations** (system instructions + reasoning traces).
2. Train student to generate both the final answer AND intermediate reasoning.
3. The student learns *reasoning patterns*, not just output distributions.

Orca-2 adds **prompt erasure**: train the student to perform well even when system prompts are removed, teaching robust reasoning strategies.

7.6.3 In-Context Distillation

Generate training data by prompting a large model with few-shot examples, then fine-tune a small model on the generated data. Variants:

- **Alpaca-style (Self-Instruct):** Use the teacher to generate instruction-response pairs from seed tasks.
- **WizardLM:** Evol-Instruct—iteratively refine instructions through the teacher to increase complexity and diversity.
- **UltraChat/UltraFeedback:** Use teacher to generate multi-turn conversations, then filter for quality.

7.6.4 Progressive Module Distillation

Distill transformer blocks one at a time rather than the full model simultaneously (Liang et al., 2023). The student's layer i is trained to match teacher's layer j ($j > i$) incrementally, reducing the complexity gap.

7.6.5 LaMini-LM (Wu et al., 2023)

Two-stage LLM distillation:

1. **Knowledge distillation:** Train student to match teacher's token-level distribution.
2. **Instruction tuning:** Fine-tune on instruction-following data generated by teacher.

Produces strong 1B–3B chat models from 70B+ teachers.

7.6.6 MoE Distillation (DeepSeek-V2 style)

Distill experts from a dense teacher into a Mixture of Experts student. The teacher's FFN layers are analyzed to discover natural specialization patterns, then mapped to expert routing in the student.

7.7 Quantization-Aware Distillation

Combine distillation with quantization:

- **Q-Distillation:** Teacher is full-precision; student is both smaller AND quantized. Soft targets partially compensate for quantization error.
- **LLM-QAT:** Jointly apply distillation from FP teacher + quantization-aware training. State-of-the-art for 4-bit student LLMs.

- **BitDistiller (2024):** Asymmetric distillation where student uses lower precision than teacher at every layer, with learned dequantization functions.

Chapter 8

Parameter-Efficient Fine-Tuning (PEFT)

PEFT methods adapt large models to new tasks by training only a small fraction of parameters, making fine-tuning feasible on consumer hardware.

8.1 LoRA: Low-Rank Adaptation (Hu et al., 2022)

8.1.1 The Low-Rank Hypothesis

The **change** $\Delta\mathbf{W}$ in weights during fine-tuning has low "intrinsic rank." The weight update can be factorized:

$$\mathbf{W}_{\text{adapted}} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA} \tag{8.1}$$

where $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ is frozen, $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$ (typically 4–64).

8.1.2 Forward Pass

For input $\mathbf{x}$, the adapter output is:

$$\mathbf{h} = \mathbf{W}_0\mathbf{x} + \frac{\alpha}{r} \cdot \mathbf{BAx} \tag{8.2}$$

where α is a scaling hyperparameter (typically $2\times r$, e.g., $\alpha = 16$ for $r = 8$). The forward pass overhead is $O(r(d + k))$, negligible compared to the base $O(dk)$.

8.1.3 Initialization

$\mathbf{A}$: Kaiming/random Gaussian initialization. $\mathbf{B}$: Zeros, so adapter starts as identity (no modification to pre-trained model).

8.1.4 Target Modules

In transformers, LoRA is typically applied to $\mathbf{W}_Q$ and $\mathbf{W}_V$ (attention pattern adaptation). Optionally $\mathbf{W}_K, \mathbf{W}_O, \mathbf{W}_{\text{up}}, \mathbf{W}_{\text{down}}$ for more expressivity. Adapting all linear layers provides best performance at higher parameter cost.

8.1.5 LoRA Code

Listing 8.1: LoRA with PEFT library

```
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8, lora_alpha=32, lora_dropout=0.1,
    target_modules=["q_proj", "v_proj"],
)
model = get_peft_model(base_model, lora_config)
# Output: trainable: 4.2M / 7.0B (0.06%)
```

8.1.6 LoRA Variants

- **LoRA+:** Uses different learning rates for $\mathbf{A}$ and $\mathbf{B}$ ($\eta_B = \lambda \cdot \eta_A$, $\lambda \gg 1$). Improves feature learning dynamics.
- **DyLoRA:** Trains multiple ranks simultaneously within a budget, dynamically selecting the optimal rank per task.
- **MixLoRA:** Uses mixture-of-experts routing to select different LoRA adapters per token.
- **S-LoRA (Service-LoRA):** Efficient multi-LoRA serving system for handling thousands of concurrently adapted models.

8.2 QLoRA: Quantized LoRA (Dettmers et al., 2023)

8.2.1 Enabling 65B Fine-Tuning on 48GB

QLoRA combines four innovations:

1. **NF4 quantization** of the frozen base model.
2. **LoRA adapters** in BF16 precision (trained normally).
3. **Double quantization:** Quantize the quantization scales to FP8.
4. **Paged optimizers:** CPU offloading for optimizer states via unified memory.

8.2.2 Memory Analysis (7B model)

Table 8.1: Memory comparison: Full FT vs QLoRA

Component	Full FT (FP32)	QLoRA (NF4)
Weights	28 GB	3.5 GB
Gradients	28 GB	0 (frozen base)
Optimizer (Adam)	56 GB	~33 MB
Activations	~5 GB	~1 GB
Total	~117 GB	~4.5 GB

Listing 8.2: QLoRA setup

```
from transformers import BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    quantization_config=bnb_config, device_map="auto",
)
# Then apply LoRA config as shown above
```

8.3 AdaLoRA: Adaptive Rank Allocation (Zhang et al., 2023)

8.3.1 Limitation of Fixed-Rank LoRA

Same rank r for all weight matrices, but different layers/heads may require different adaptation capacity. AdaLoRA dynamically allocates the parameter budget.

8.3.2 SVD-Based Parameterization

Parameterizes the update as a truncated SVD:

$$\Delta \mathbf{W} = \mathbf{P}\mathbf{\Lambda}\mathbf{Q} \tag{8.3}$$

where $\mathbf{P} \in \mathbb{R}^{d\times r}$, $\mathbf{Q} \in \mathbb{R}^{r\times k}$ are approximately orthonormalized (via regularizer), and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_r)$.

Importance of each rank component:

$$I(\lambda_i) = |\lambda_i| \cdot \|\nabla_{\lambda_i} \mathcal{L}\| \tag{8.4}$$

Periodically: prune least important singular values, reallocate budget to more important ones (grow their rank).

8.4 DoRA: Weight-Decomposed Low-Rank Adaptation (Liu et al., 2024)

8.4.1 Learning Magnitude and Direction Separately

Decomposes pre-trained weights into **magnitude** and **direction**:

$$\mathbf{W}_{\text{adapted}} = \mathbf{m} \cdot \frac{\mathbf{W}_0 + \mathbf{BA}}{\|\mathbf{W}_0 + \mathbf{BA}\|_c} \tag{8.5}$$

where $\mathbf{m} \in \mathbb{R}^{d\times 1}$ is a learnable magnitude vector (initialized as $\|\mathbf{W}_0\|_c$), and $\|\cdot\|_c$ is column-wise L2 norm. DoRA learns to *rotate* each column ($\mathbf{BA}$) and *scale* it ($\mathbf{m}$), achieving the expressivity of full fine-tuning in the limit.

8.5 VeRA: Vector-based Random Matrix Adaptation (Kopiczko et al., 2024)

Uses frozen random matrices shared across all layers:

$$\Delta\mathbf{W} = \mathbf{\Lambda}_b \cdot \mathbf{B}_{\text{shared}} \cdot \mathbf{A}_{\text{shared}} \cdot \mathbf{\Lambda}_d \tag{8.6}$$

where $\mathbf{A}_{\text{shared}}, \mathbf{B}_{\text{shared}}$ are frozen random matrices (one pair for the entire model), and only diagonal scaling vectors $\mathbf{\Lambda}_b, \mathbf{\Lambda}_d$ are trained. Reduces trainable parameters by 10× compared to LoRA.

8.6 $(IA)^3$: Infused Adapter (Liu et al., 2022)

8.6.1 Rescaling Vectors

Trains vectors that rescale transformer internals element-wise:

$$\mathbf{Q}' = (l_q \odot \mathbf{Q}), \quad \mathbf{K}' = (l_k \odot \mathbf{K}), \quad \mathbf{V}' = (l_v \odot \mathbf{V}) \tag{8.7}$$

$$\text{FFN}' = (l_{ff} \odot \gamma(\mathbf{x}\mathbf{W}_1))\mathbf{W}_2 \tag{8.8}$$

where $l_q, l_k, l_v, l_{ff} \in \mathbb{R}^d$ are learned vectors. Only ∼0.01% of parameters trained. Strong few-shot performance.

8.7 Prefix Tuning and Prompt Tuning

8.7.1 Prefix Tuning (Li & Liang, 2021)

Prepends learnable continuous vectors at every transformer layer:

$$\mathbf{H}^{(\ell)} = \text{TransformerLayer}_\ell([\underbrace{\mathbf{P}_1^{(\ell)}, \ldots, \mathbf{P}_m^{(\ell)}}_{\text{prefix}}, \mathbf{H}^{(\ell-1)}]) \tag{8.9}$$

$\mathbf{P}^{(\ell)}$ are reparameterized via a small MLP: $\mathbf{P}^{(\ell)} = \text{MLP}_\ell(\mathbf{P}_\theta)$ for training stability.

8.7.2 Prompt Tuning (Lester et al., 2021)

Adds learnable tokens only at the input embedding layer:

$$\mathbf{H}^{(0)} = [\mathbf{p}_1, \ldots, \mathbf{p}_m, \text{Embed}(x_1), \ldots, \text{Embed}(x_n)] \tag{8.10}$$

Soft prompts—continuous vectors optimized end-to-end. Less expressive than prefix tuning but more parameter-efficient.

8.7.3 P-Tuning v2

Extends prompt tuning to **deep prompts** at every layer (like prefix tuning) with residual connections. More expressive, especially for NLU tasks.

8.8 Additional PEFT Variants

8.8.1 LoHA (Low-Rank Hadamard Product)

Uses element-wise product of two low-rank matrices:

$$\Delta\mathbf{W} = (\mathbf{B}_1\mathbf{A}_1) \odot (\mathbf{B}_2\mathbf{A}_2) \tag{8.11}$$

Can achieve higher effective rank with same parameter count.

8.8.2 LoKr (Low-Rank Kronecker Product)

Uses Kronecker product decomposition:

$$\Delta\mathbf{W} = \mathbf{C} \otimes (\mathbf{BA}) \tag{8.12}$$

Exploits tensor structure for efficient compression.

8.8.3 OFT / BOFT (Orthogonal Fine-Tuning)

Trains an orthogonal transformation (rotation matrix) applied to pre-trained weights. BOFT uses butterfly matrices for efficient parameterization of the orthogonal group, achieving $O(d \log d)$ parameters instead of $O(d^2)$.

8.8.4 SSF (Scaling and Shifting Features)

Per-layer per-channel scale $\boldsymbol{\gamma}$ and shift $\boldsymbol{\beta}$:

$$\mathbf{y}' = \boldsymbol{\gamma} \odot \mathbf{y} + \boldsymbol{\beta} \tag{8.13}$$

Extremely lightweight: only $2d$ parameters per adapted layer.

8.8.5 KronA (Kronecker Adapter)

Applies Kronecker-factored weight updates: $\Delta\mathbf{W} = \mathbf{W}_{\text{down}} \otimes \mathbf{W}_{\text{up}}$, where each factor is low-rank. Leverages the mathematical properties of Kronecker products for structured adaptation.

8.8.6 MoRA (Mixture of Rank-1 Adapters)

Combines rank-1 adapters with router selection, enabling different rank-1 adaptations for different domains within the same model.

8.9 PEFT Comparison

Table 8.2: Comparison of PEFT methods (7B model)

Method	**Trainable**	**% Total**	**Mechanism**
Full Fine-Tuning	7.0B	100%	All weights updated
LoRA ($r = 8$)	4.2M	0.06%	Low-rank decomposition
QLoRA ($r = 16$, NF4)	8.4M	0.12%	Quantized base + LoRA
$(IA)^3$	0.8M	0.01%	Learned rescaling vectors
Prefix Tuning	1.2M	0.02%	Per-layer virtual tokens
Prompt Tuning	20K	0.0003%	Input-layer soft prompts
DoRA ($r = 8$)	4.3M	0.06%	Magnitude/direction split
VeRA ($r = 256$)	0.4M	0.006%	Shared frozen random mats
AdaLoRA	4.2M	0.06%	Adaptive rank allocation

8.10 PEFT Best Practices

- **Target Q + V** for most tasks (best performance/parameter tradeoff).
- Add **K and O** for complex tasks requiring more expressivity.
- **Rank** $r = 8$–16 sufficient for most use cases; higher rank for math/reasoning.
- **Merge adapters** into base weights after training for inference (no overhead).
- **Load/Serve multiple LoRAs** simultaneously for multi-task inference.

Chapter 9

Preference Alignment Techniques

Aligning LLMs with human preferences is critical for producing helpful, harmless, and honest outputs. This chapter covers the full spectrum of alignment methods from RLHF to reference-free approaches.

9.1 RLHF: Reinforcement Learning from Human Feedback

9.1.1 Three-Stage Pipeline (Christiano et al., 2017; Ouyang et al., 2022)

Stage 1—SFT (Supervised Fine-Tuning): Fine-tune the pre-trained model on high-quality demonstration data (prompt, ideal response pairs).

Stage 2—Reward Model (RM) Training: Collect human preference data: for each prompt, two responses y_w (preferred/won) and y_l (dispreferred/lost). Train $r_\phi(x, y)$ using the Bradley-Terry model:

$$P(y_w \succ y_l \mid x) = \frac{\exp(r_\phi(x, y_w))}{\exp(r_\phi(x, y_w)) + \exp(r_\phi(x, y_l))} \tag{9.1}$$

The RM loss is binary cross-entropy:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)}\big[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\big] \tag{9.2}$$

Stage 3—PPO Optimization: Optimize the SFT policy π_θ against the reward model using PPO (Schulman et al., 2017):

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x, y) - \beta \cdot \text{KL}(\pi_\theta(\cdot|x)\|\pi_{\text{ref}}(\cdot|x))\big] \tag{9.3}$$

The KL penalty prevents divergence from the reference policy π_{ref} (the SFT model).

9.1.2 PPO Mechanic

The clipped surrogate objective:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t\big[\min(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\big] \tag{9.4}$$

where $\rho_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the advantage estimate. RLHF requires maintaining four models simultaneously (policy, reference, reward, value), making it extremely memory-intensive.

9.1.3 RLHF Variants

- **REINFORCE with baseline:** Simpler than PPO, uses full-sequence sampling without clipping.
- **GRPO (Group Relative Policy Optimization):** Compare multiple sampled responses per prompt against each other, eliminating the need for a separate value model.
- **ReMax:** Uses the reward of the best sample as a constant baseline, simplifying REINFORCE.
- **RAFT (Reward rAnked Fine-Tuning):** Sorts multiple samples by reward and fine-tunes on the top-k only (simpler than PPO).

9.2 DPO: Direct Preference Optimization (Rafailov et al., 2023)

9.2.1 Eliminating the Reward Model

DPO derives an analytical mapping between the optimal policy under KL-constrained RLHF and the reward function:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \tag{9.5}$$

Substituting into the Bradley-Terry model and canceling the partition function $Z(x)$:

$$P(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right) \tag{9.6}$$

This yields a direct loss on preference pairs:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\Big] \tag{9.7}$$

9.2.2 Gradient Analysis

The DPO gradient reveals its behavior:

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E}_{(x,y_w,y_l)}\big[\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)) \cdot (\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_l|x))\big] \tag{9.8}$$

It **increases** log-probability of preferred responses and **decreases** it for dispreferred ones, weighted by how incorrectly the current policy rates them (higher weight when the policy gets it wrong).

9.2.3 DPO Variants

- **Conservative DPO (cDPO):** Adds a smoothness regularizer preventing probability collapse.
- **Iterative DPO:** Multiple rounds of DPO training with online-generated preference data.

- **Batched DPO:** Efficient implementation enabling training on preference pairs from different prompts in a single batch.
- **cDPO (contrastive DPO):** Uses contrastive learning principles with negative examples.
- **DPOP (DPO with Positive-negative constraint):** Prevents DPO from reducing the probability of both responses simultaneously.

9.3 IPO: Identity Preference Optimization (Azar et al., 2024)

9.3.1 Addressing DPO Limitations

IPO avoids the Bradley-Terry assumption and DPO's log-sigmoid saturation:

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}_{(x,y_w,y_l)}\left[\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{\tau^{-1}}{2}\right)^2\right] \tag{9.9}$$

where τ is a regularization parameter. Advantages:

- Avoids assumptions about the underlying preference model.
- Less prone to overfitting (no log-sigmoid saturation as probabilities approach 0 or 1).
- Stronger theoretical convergence guarantees under the Ψ-Preference Optimization framework.

9.4 KTO: Kahneman-Tversky Optimization (Ethayarajh et al., 2024)

9.4.1 Prospect Theory Motivation

Inspired by Kahneman & Tversky's observation that humans are **loss-averse**: the pain of a loss outweighs the pleasure of an equivalent gain.

9.4.2 No Paired Data Required

KTO does not require paired preference data (chosen vs rejected for the same prompt). It only needs binary desirable/undesirable labels:

$$\mathcal{L}_{\text{KTO}} = \mathbb{E}_{(x,y,\text{label})}\left[\lambda_D \cdot \ell_D(y_{\text{desirable}}) + \lambda_U \cdot \ell_U(y_{\text{undesirable}})\right] \tag{9.10}$$

The weighting $\lambda_D < \lambda_U$ reflects loss-aversion (undesirable outputs are penalized more heavily than desirable ones are rewarded). This simplifies data collection significantly.

9.5 ORPO: Odds Ratio Preference Optimization (Hong et al., 2024)

9.5.1 Unified SFT + Alignment

ORPO combines SFT and preference alignment into one stage:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda \cdot \mathcal{L}_{\text{OR}} \tag{9.11}$$

The odds ratio loss penalizes the model for assigning high probability to dispreferred responses:

$$\mathcal{L}_{\text{OR}} = -\log \sigma\left(\log \frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)}\right) \tag{9.12}$$

where $\text{odds}_\theta(y|x) = \frac{\pi_\theta(y|x)}{1-\pi_\theta(y|x)}$. No reference model needed, making it reference-free and simpler than DPO.

9.6 SimPO: Simple Preference Optimization (Meng et al., 2024)

9.6.1 Reference-Free with Length Normalization

Completely eliminates the reference model:

$$\mathcal{L}_{\text{SimPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l|x) - \gamma\right)\right] \tag{9.13}$$

Key differences from DPO:

- **No reference model:** Reward is the average log-probability $\frac{1}{|y|}\sum_t \log \pi_\theta(y_t|x, y_{<t})$.
- **Length normalization:** Prevents bias toward shorter (often less informative) responses.
- **Target reward margin γ:** Encourages a minimum gap between preferred and dispreferred rewards.

9.7 RLAIF: RL from AI Feedback (Bai et al., 2022)

9.7.1 Constitutional AI

Replaces humans in the feedback loop with an AI judge:

Stage 1—Supervised:

1. Generate responses to harmful prompts.
2. AI judge critiques responses according to a **constitution** (principles like "be helpful," "be harmless").
3. AI judge revises responses based on the critique.
4. Fine-tune on revised responses.

Stage 2—RL:

1. Generate response pairs from the fine-tuned model.
2. AI judge provides preference labels based on the constitution.

3. Train preference model on AI-generated preferences.
4. Use RL (PPO) with the PM as reward.

9.7.2 UltraFeedback (Cui et al., 2023)

Uses GPT-4 to rate and critique model responses across multiple dimensions (helpfulness, honesty, harmlessness, instruction-following), producing preference data for DPO/RLHF training without human annotators.

9.8 RLVR: RL with Verifiable Rewards (Lambert et al., 2024)

9.8.1 Outcome-Based Binary Rewards

For tasks with objectively verifiable outcomes (math, code):

$$r(x, y) = \begin{cases} 1 & \text{if response passes verification} \\ 0 & \text{otherwise} \end{cases} \tag{9.14}$$

Verification methods: exact answer match for math, compilation + test pass for code, factuality check for QA. Eliminates reward hacking concerns entirely.

9.9 CPO: Contrastive Preference Optimization (Xu et al., 2024)

Frames alignment as contrastive learning:

$$\mathcal{L}_{\text{CPO}} = -\mathbb{E}\big[\log \frac{e^{\pi_\theta(y_w|x)/\tau}}{e^{\pi_\theta(y_w|x)/\tau} + \sum_{y_i \in \mathcal{N}} e^{\pi_\theta(y_i|x)/\tau}}\big] \tag{9.15}$$

where $\mathcal{N}$ is a set of negative (dispreferred) responses. The contrastive framework naturally models relative preferences.

9.10 SPIN: Self-Play Fine-Tuning (Chen et al., 2024)

The model generates its own training data through self-play:

1. Current model generates responses to prompts.
2. Previous model iteration serves as opponent.
3. Train to distinguish and improve over the opponent.
4. Iterate until convergence.

Requires no external preference data—bootstraps alignment from the model's own generations.

9.11 Comparison of Alignment Methods

Table 9.1: Preference alignment method comparison

Method	RM	Ref.	Paired	Stability	Compute
RLHF (PPO)	Yes	Yes	Yes	Medium	Very High
DPO	No	Yes	Yes	High	Low
IPO	No	Yes	Yes	Very High	Low
KTO	No	Yes	No	High	Low
ORPO	No	No	Yes	High	Very Low
SimPO	No	No	Yes	High	Very Low
RLAIF	Yes (AI)	Yes	Yes	Medium	High
RLVR	No (rule)	Yes	Yes	High	Medium
SPIN	No	Self	Yes	High	Medium

9.12 Practical Recommendations

- **Small-scale alignment (hobbyist):** ORPO or SimPO—no reference model needed, single-stage training.
- **With preference data:** DPO with $\beta = 0.1$ for best quality/compute ratio.
- **Without paired data:** KTO with binary good/bad labels.
- **With strong teacher available:** RLAIF using GPT-4 or Claude as judge.
- **Math/code domains:** RLVR with verifiable binary rewards.
- **Self-improving loop:** SPIN for continuous self-play refinement.

Chapter 10

Mixture of Experts (MoE)

MoE models activate only a subset of parameters per token, dramatically scaling model capacity while keeping inference FLOPs approximately constant.

10.1 MoE Architecture Fundamentals

10.1.1 Core Idea

Instead of one FFN per transformer layer, maintain E experts (each an independent FFN). For each input token, a **router** (gating network) selects the top-k experts:

$$\mathbf{g} = \text{softmax}(\mathbf{x}\mathbf{W}_g) \quad \text{(routing logits)} \tag{10.1}$$

$$\text{TopK}(\mathbf{g}, k) = \text{keep top } k\text{, set others to 0} \tag{10.2}$$

$$\mathbf{y} = \sum_{i \in \text{selected}} g_i \cdot \text{FFN}_i(\mathbf{x}) \tag{10.3}$$

$$\mathbf{y}_{\text{final}} = \mathbf{y} + \mathbf{x} \quad \text{(residual from attention)} \tag{10.4}$$

Common: $k = 1$ (Switch Transformer), $k = 2$ (Mixtral, GShard). Higher k = more capacity, more compute.

10.1.2 Load Balancing

Without balancing, the router collapses to using a few experts. Two common solutions:

Auxiliary loss (Switch Transformer):

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_{i=1}^{E} f_i \cdot P_i \tag{10.5}$$

where $f_i = \frac{1}{T}\sum_t \mathbf{1}[\text{token } t \to \text{expert } i]$ (fraction of tokens) and $P_i = \frac{1}{T}\sum_t g_{i,t}$ (fraction of router probability). Minimized when both are uniform.

Z-loss (training stability):

$$\mathcal{L}_z = \frac{1}{T}\sum_t \left(\log \sum_i e^{z_{i,t}}\right)^2 \tag{10.6}$$

Penalizes large router logits, preventing numerical instability.

10.1.3 Expert Capacity

Each expert processes at most C tokens:

$$C = \left\lceil \frac{\text{tokens}}{E} \cdot \text{capacity factor} \right\rceil \tag{10.7}$$

Excess tokens are **dropped** (output set to the residual connection). Capacity factor > 1 reduces dropping at compute cost. Typical: 1.25.

10.2 Major MoE Implementations

10.2.1 Switch Transformer (Fedus et al., 2022)

$k = 1$: one expert per token. Demonstrates that single-expert routing is sufficient—different tokens route to different experts, achieving similar quality to $k = 2$ with simpler communication.

10.2.2 GShard (Lepikhin et al., 2021)

$k = 2, E = 2048$: massive expert count for extreme parameter scaling. Introduces **expert parallelism**: each expert on a different device. Tokens dispatched via all-to-all communication.

10.2.3 Mixtral 8×7B (Jiang et al., 2024)

$E = 8$ experts, $k = 2$ per token. 46.7B total, 12.9B active. Replaces FFN in every *other* transformer layer with MoE (dense layers between maintain representational continuity). Outperforms LLaMA-2 70B with $\sim 5\times$ fewer active parameters.

10.2.4 DeepSeek-MoE (Dai et al., 2024)

Key innovations:

- **Shared experts** (always activated) + **routed experts** (gated). Shared experts handle common patterns; routed experts specialize.
- **Fine-grained experts:** More experts with smaller hidden dimension.
- **Expert-level load balancing:** Device-aware auxiliary losses.

10.2.5 Qwen2-MoE / dMoE

Upcycling from dense: Initialize MoE from dense checkpoints by replicating FFN weights (with noise) and adding routers. Reduces MoE pre-training cost by 40–60%.

10.2.6 OpenMoE / JetMoE

Open-source MoE implementations with $E = 8$, trained from scratch or upcycled. Demonstrate that MoE can be cost-effective even at smaller scales (1B–7B active parameters).

10.3 Expert Routing Strategies

10.3.1 Token-Choice Routing (Standard)

Each token independently chooses its top-k experts. Simple, expressive, but prone to load imbalance.

10.3.2 Expert-Choice Routing (Zhou et al., 2022)

Each expert selects its top-C tokens. Guarantees perfect load balance but some tokens may be unselected (dropped) or multiply selected. Often combined with token-choice as a hybrid.

10.3.3 Hash Routing (Roller et al., 2021)

Deterministic routing based on a hash of the token ID. Load-balanced by construction (hash uniformity), but not content-adaptive—routes similar tokens to different experts.

10.3.4 Soft MoE (Puigcerver et al., 2023)

Instead of hard routing, computes a weighted combination of **all** experts:

$$\mathbf{y} = \sum_{i=1}^{E} \phi(\mathbf{x})_i \cdot \mathrm{FFN}_i(\mathbf{x}) \tag{10.8}$$

where $\phi(\mathbf{x}) = \mathrm{softmax}(\mathbf{x}\mathbf{W}_\phi)$. Fully differentiable, no drop. But activates all experts at inference (losing efficiency).

10.3.5 DeepSeek-V2 Routing

Uses **device-level** and **expert-level** auxiliary losses simultaneously, with shared + routed expert decomposition and group-level top-k gating for efficient fine-grained expert selection.

10.4 MoE for Attention (MoA)

Apply the MoE principle to attention heads: only a subset of attention heads activate per token. The routing mechanism selects which heads compute attention based on input content. Mathematically:

$$\mathbf{o} = \sum_{i \in \text{selected}} g_i \cdot \mathrm{AttentionHead}_i(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i) \tag{10.9}$$

Combines the capacity scaling of MoE with the contextual processing of attention.

10.5 PEER: Parameter Efficient Expert Retrieval (He et al., 2024)

Uses embedding-based expert retrieval instead of learned router:

1. Each expert has an embedding key $\mathbf{e}_i \in \mathbb{R}^d$.
2. Input $\mathbf{x}$ generates a query $\mathbf{q} = f_{\text{query}}(\mathbf{x})$.
3. Top-k experts are retrieved via nearest-neighbor search: $\mathbf{q} \cdot \mathbf{e}_i$.

Enables 10^6+ experts (only top-k activated) while keeping routing cost constant via efficient approximate NN search. Each expert is a tiny MLP. Parameter-efficient: majority of parameters are in the expert database.

10.6 MoE Inference Considerations

10.6.1 Memory Requirements

All experts must reside in memory, even if only k are activated per token. For Mixtral 8×7B: all 8 experts (∼93 GB FP16) vs. active 12.9B.

Mitigations:

- **Expert offloading:** Keep frequently-used experts in VRAM, offload others to CPU RAM, swap as needed.
- **Expert quantization:** Quantize all experts (QMoE, QFQ), substantially reducing total memory footprint.
- **Expert pruning:** Drop experts with consistently low utilization.

10.6.2 Expert Parallelism Communication

When experts are distributed across GPUs:

1. Each device sends tokens to devices hosting selected experts (all-to-all send).
2. Devices compute expert outputs.
3. Results sent back to original devices (all-to-all recv).

Communication overhead limits throughput at small batch sizes. Large batches amortize the cost.

10.6.3 Expert Caching

Cache expert choices for repeated token patterns (e.g., common words always route to the same experts). Can skip router computation for cached tokens, reducing overhead.

10.7 MoE Training Stability Techniques

- **Dense initialization (upcycling):** Start from a dense checkpoint, reducing MoE training instability.
- **Router z-loss:** Prevents exploding router logits during early training.
- **Expert dropout:** Randomly drop some experts during training to encourage robustness.

- **Load-balanced router initialization:** Initialize $\mathbf{W}_g$ to produce uniform routing at the start.
- **Gradient clipping on router:** Separate clipping thresholds for router gradient norms.

Chapter 11

Inference Engine Selection and Hardware Optimization

Choosing the right inference engine and optimizing for specific hardware is critical for cost-effective local deployment.

11.1 Major Inference Engines

11.1.1 llama.cpp (CPU, Edge, and GPU via BLAS)

Focus: CPU inference, Apple Silicon (Metal), and edge devices. **Format:** GGUF model files. **Key features:**

- KV-cache quantization (Q8_0, Q4_0)—4× memory savings.
- Memory-mapped (mmap) loading—near-instant load times.
- BLAS backends: OpenBLAS, Intel MKL, cuBLAS, CLBlast, Vulkan.
- GPU offloading: −ngl N layers to GPU, rest on CPU.
- 'llama-server': HTTP API with continuous batching, slot-based request management.
- Speculative decoding via draft models.
- Grammar-constrained generation (GBNF grammars).

Listing 11.1: llama.cpp server with GPU offloading

```
./llama-server -m model-Q4_K_M.gguf \
  -ngl 33 -c 4096 --mlock --threads 8 \
  --port 8080
```

11.1.2 vLLM (GPU Server, Kwon et al., 2023)

Focus: Maximum GPU throughput via PagedAttention. **Key features:**

- PagedAttention KV-cache management (near-zero memory waste).
- Continuous batching (dynamic request addition/removal).
- Tensor parallelism across multiple GPUs.
- Prefix caching: shared prefixes (e.g., system prompts) computed once.
- Quantization: AWQ, GPTQ, FP8, SqueezeLLM, Marlin kernels.
- Speculative decoding.
- 100+ supported model architectures.

Listing 11.2: vLLM offline inference

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="mistralai/Mistral-7B-Instruct-v0.2",
    tensor_parallel_size=2,
    gpu_memory_utilization=0.9,
    max_model_len=8192,
)
outputs = llm.generate(["Explain quantum computing"],
    SamplingParams(temperature=0.7, max_tokens=256))
```

11.1.3 Text Generation Inference (TGI, HuggingFace)

Focus: Production-grade serving with security. **Key features:**

- Watermarking generated text for content attribution.
- Safety filtering for harmful content detection.
- Tensor Parallelism and Paged Attention.
- Quantization: bitsandbytes, GPTQ, AWQ, EETQ.
- Guided generation via outlines (JSON, regex, grammar constraints).
- Request queuing, dynamic batching, and token streaming.

11.1.4 TensorRT-LLM (NVIDIA)

Focus: Maximum NVIDIA GPU performance via graph-level optimization. **Key features:**

- **Graph optimization:** Fuses multiple operations into single CUDA kernels (RMSNorm + quantize + matmul + dequantize).
- In-flight batching (NVIDIA's term for continuous batching).
- FP8, INT4, INT8 quantization with optimized kernels.
- Multi-GPU: Tensor Parallelism + Pipeline Parallelism.
- Speculative decoding via draft model or Medusa heads.
- Plugin-based extensibility for custom kernels.
- 2–5× faster than vanilla PyTorch/HF transformers.

11.1.5 SGLang (Structured Generation Language)

Focus: Structured/constrained generation with RadixAttention. **Key features:**

- RadixAttention: prefix caching via a radix tree structure for maximum prefix sharing.
- Native constrained generation (JSON, regex, grammars).
- Python-native control flow in generation.
- Continuous batching similar to vLLM.

11.1.6 MLX (Apple Silicon)

Apple's native ML framework for M-series chips:

- **Unified memory architecture:** No CPU-GPU copies. Weights, KV-cache, activations share one memory pool.
- Lazy evaluation with computation graph optimization.
- Competitive with llama.cpp Metal backend; Python-native API.

Listing 11.3: MLX inference

```
from mlx_lm import load, generate
model, tokenizer = load("mlx-community/Mistral-7B-v0.2-4bit")
response = generate(model, tokenizer, prompt="Hello",
    max_tokens=100)
```

11.1.7 Ollama

User-friendly wrapper around llama.cpp:

Listing 11.4: Ollama usage

```
ollama run llama3.1:8b
ollama create mymodel -f Modelfile
# Modelfile: FROM llama3.1:8b
#   PARAMETER temperature 0.7
#   SYSTEM You are a helpful assistant
```

11.1.8 Other Notable Engines

- **ExLlamaV2:** GPU-only, extremely optimized GPTQ/EXL2 inference with fused dequantization kernels.
- **Aphrodite Engine:** vLLM fork optimized for roleplay/chat applications.
- **Candle (Rust):** Minimalist Rust ML framework for WASM, edge devices, and server inference.
- **OpenVINO (Intel):** Optimized for Intel CPUs, iGPUs, and NPUs with INT8/FP16 inference.
- **MNN (Alibaba):** Lightweight engine for mobile/embedded devices.
- **PowerInfer:** Hot-cold neuron offloading for consumer GPUs.

11.2 Hardware-Specific Optimizations

11.2.1 CUDA Kernel Optimization

- **FlashInfer:** Library of optimized CUDA kernels for LLM inference (attention, sampling, positional encoding). Used by vLLM, SGLang.
- **CUTLASS:** NVIDIA template library for high-performance GEMM.
- **Triton:** Python DSL for writing GPU kernels. Flash Attention-2 implemented in Triton. Reaches 90–95% of hand-tuned CUDA performance.
- **Marlin kernels (IST-DASLab):** Extremely optimized INT4 matmul kernels for GPTQ models, delivering near-memory-bandwidth utilization.
- **Fused kernels:** Combine RMSNorm + RoPE + QKV projection into single kernel launch.

11.2.2 CPU Optimization

- **Intel AMX (Advanced Matrix Extensions):** Accelerates INT8/BF16 matmul on Sapphire Rapids+. llama.cpp supports via oneDNN backend.
- **ARM NEON / SVE:** SIMD instructions for ARM CPUs (Apple M-series, Qualcomm, AWS Graviton).
- **Apple AMX:** Proprietary matrix coprocessor on M1+ for matmul acceleration (used by MLX, CoreML).
- **RVV (RISC-V Vector):** Emerging support for RISC-V vector extensions.

11.2.3 GPU Memory Optimization

- **FlashAttention:** Minimizes HBM reads.
- **KV-cache quantization:** K and V stored in INT8/FP8, reducing KV-cache memory 2–4×.
- **GQA/MQA:** Architectural KV-cache reduction.
- **Prefix caching:** Share KV-cache for common prefixes across requests.
- **Automatic KV-cache eviction:** LRU/adaptive eviction policies when cache is full.

11.3 Multi-GPU Deployment Strategies

11.3.1 Tensor Parallelism (TP)

Split each weight matrix across GPUs along the column dimension:

$$\mathbf{Y} = \mathbf{X}[\mathbf{W}_1 \| \cdots \| \mathbf{W}_P] = [\mathbf{X}\mathbf{W}_1 \| \cdots \| \mathbf{X}\mathbf{W}_P] \tag{11.1}$$

After matmul: all-reduce combines results. Requires high-bandwidth interconnects (NVLink, PCIe 4.0+). Communication on every forward pass.

LLaMA TP: Attention and FFN projections are column-partitioned. RMSNorm, RoPE, element-wise ops are replicated. Synchronized all-reduce after each partitioned matmul.

11.3.2 Pipeline Parallelism (PP)

Split model by layers across GPUs. Computation proceeds as a pipeline with micro-batches:

- **GPipe (F-then-B):** All forwards, then all backwards. Largest bubbles.
- **1F1B:** Alternate forward and backward per micro-batch. Balances activation memory and bubbles.
- **Interleaved 1F1B:** Non-contiguous layers per GPU. Minimizes bubble overhead.

11.3.3 Data Parallelism (DP) for Inference

Replicate full model across GPUs, each serving independent requests. Throughput scales linearly with GPU count.

11.3.4 ZeRO-Inference (DeepSpeed)

Partitions weights across GPUs (ZeRO-3 style). Before each layer: all-gather weights, compute, discard. Enables models larger than any single GPU's memory at the cost of communication overhead.

11.3.5 Sequence Parallelism

Split long sequences across GPUs (Ring Attention or Striped Attention). Essential for ultra-long context (>100K tokens). Communication volume grows linearly with seqlen.

11.4 Offloading Techniques

11.4.1 CPU Offloading

When model exceeds GPU VRAM:

- **Layer-wise:** First k layers on GPU, rest on CPU. Data transferred at boundary. llama.cpp: $-ngl N$.
- **Decode-phase offloading:** Keep full model on GPU during prefill (compute-bound), offload layers to CPU during decode (memory-bound).
- **PowerInfer:** Hot neurons (frequently activated) on GPU; cold neurons on CPU. Exploits activation sparsity.

11.4.2 Disk Offloading (FlexGen, Sheng et al., 2023)

Offload to SSD/NVMe for extreme memory constraints:

1. **Block-schedule:** Organize computation into blocks (layers, KV-cache chunks).
2. **Linear programming solver:** Find optimal schedule: GPU/CPU/SSD placement + transfer timing.
3. **Double-buffering:** Overlap I/O with computation: while GPU computes block A, I/O loads block B.

Enables OPT-175B inference at $\sim$1 tok/s on a single 16GB T4 GPU.

11.4.3 Attention Offloading (AOffload)

Offload attention computation to CPU while keeping FFN on GPU, exploiting that attention is more latency-tolerant (matmul-bound) while FFN benefits more from GPU parallelism.

11.5 Batching and Scheduling

11.5.1 Continuous (In-flight) Batching

Dynamically add new requests to running batches and remove completed ones per-token:

1. Maintain queue of pending requests.
2. Each iteration: select subset that fits in memory.
3. Compute one decoder step.
4. Remove requests that generated EOS.

5. Add new requests from queue.

Improves GPU utilization from 30–50% (static) to 90%+ (continuous).

11.5.2 Chunked Prefill (Sarathi/Splitwise)

Long prompts are compute-bound (quadratic attention) and can starve decode steps. Chunked prefill:

1. Process C tokens of longest pending prompt.
2. Run one decode step for ongoing generations.
3. Repeat until all prompts fully prefilled.

Prevents long prefills from blocking decode, improving time-to-first-token (TTFT) for concurrent users.

11.5.3 Priority Scheduling

- **FCFS (First-Come First-Served):** Simplest. High latency variance.
- **SJF (Shortest Job First):** Prioritize short requests. Minimizes average latency, risk of starvation.
- **Multi-level feedback queue:** Separate priority queues. Requests demoted with continued generation.
- **Weighted fair queuing:** Proportionally allocate computation to requests.

11.6 Inference Engine Selection Guide

Table 11.1: Engine selection by deployment scenario

Scenario	Recommended Engine	Why
CPU-only laptop	llama.cpp	GGUF, CPU-optimized BLAS
Apple M-series	MLX or llama.cpp	Unified memory, AMX
Single NVIDIA GPU	vLLM or TGI	PagedAttention, cont. batching
Multi-GPU server	vLLM + TP or TensorRT-LLM	Best multi-GPU throughput
Max NVIDIA perf.	TensorRT-LLM	Graph optimization + FP8
Edge/mobile	llama.cpp (Android) / MLC	Vulkan/OpenCL, small footprint
Structured output	SGLang or Outlines + vLLM	Native constrained generation
Production SaaS	TGI or vLLM + load balancer	Monitoring, watermarking, security

Chapter 12

Decoding Techniques

Decoding strategies determine how the model selects tokens during autoregressive generation. This chapter covers the full spectrum from basic to advanced techniques.

12.1 Deterministic Decoding

12.1.1 Greedy Decoding

Select the most probable token at each step:

$$\hat{y}_t = \underset{v \in V}{\operatorname{argmax}}\, p(y_t = v \mid y_{<t}, \mathbf{x}) \tag{12.1}$$

Deterministic. Fast. May produce repetitive or degenerate text (never explores alternatives). Used when consistency is paramount.

12.1.2 Beam Search

Maintain top-B most probable partial sequences (beams). At each step, expand all beams and keep top B:

$$\mathcal{B}_t = \text{Top-B}_{b \in \mathcal{B}_{t-1}, v \in V} \left[\log p(b) + \log p(v \mid b, \mathbf{x}) \right] \tag{12.2}$$

Finds higher-probability sequences than greedy. Tends toward shorter, safer (less diverse) outputs. Compute overhead: $B\times$ greedy cost.

12.1.3 Diverse Beam Search (Vijayakumar et al., 2018)

Penalizes similar beams via a diversity term:

$$\text{score}(b, v) = \log p(v \mid b) + \lambda \cdot \Delta(b \cup \{v\}, \mathcal{G}) \tag{12.3}$$

where Δ measures dissimilarity between candidate and already-selected beams in group $\mathcal{G}$ (e.g., Hamming distance over token sequences). Produces diverse output alternatives.

12.1.4 Contrastive Search (Su et al., 2022)

Selects tokens that balance probability and novelty:

$$\hat{y}_t = \underset{v \in V}{\operatorname{argmax}} \left[(1 - \alpha) \cdot p(v \mid y_{<t}) - \alpha \cdot \max_{j < t} \text{sim}(\mathbf{h}_v, \mathbf{h}_{y_j}) \right] \tag{12.4}$$

where $\mathbf{h}_v$ is the hidden state for token v and sim is cosine similarity. Reduces repetition while maintaining coherence.

12.2 Sampling-Based Decoding

12.2.1 Temperature Sampling

Scale logits before softmax to control randomness:

$$p(y_t = v) = \frac{e^{z_v/T}}{\sum_{v'} e^{z_{v'}/T}} \tag{12.5}$$

$T = 0$: greedy. $T = 1$: default. $T > 1$: more diverse (flatter). $T < 1$: more conservative (sharper).

12.2.2 Top-k Sampling (Fan et al., 2018)

Sample only from k most probable tokens, renormalizing:

$$p'(y_t = v) = \begin{cases} \frac{p(y_t=v)}{\sum_{w \in \mathcal{V}_k} p(y_t=w)} & \text{if } v \in \mathcal{V}_k \\ 0 & \text{otherwise} \end{cases} \tag{12.6}$$

Fixed k can be suboptimal: for peaked distributions, k may be too large (including noise); for flat distributions, k may be too small (restricting creativity).

12.2.3 Nucleus (Top-p) Sampling (Holtzman et al., 2020)

Sample from the smallest set whose cumulative probability exceeds p:

$$\mathcal{V}_p = \operatorname*{argmin}_{\mathcal{V}'} |\mathcal{V}'| \quad \text{s.t.} \quad \sum_{v \in \mathcal{V}'} p(y_t = v) \geq p \tag{12.7}$$

Adaptive: only a few tokens for high-confidence distributions; more tokens when uncertain. Generally produces more coherent text than fixed k.

12.2.4 Min-p Sampling (Nguyen et al., 2024)

Sample from tokens with probability at least p_{min} times the maximum:

$$\mathcal{V}_{\text{min-p}} = \{v : p(y_t = v) \geq p_{\text{min}} \cdot \max_w p(y_t = w)\} \tag{12.8}$$

Dynamically adjusts threshold based on the peak probability, preventing sampling from near-zero probability tokens while allowing full exploration when the model is uncertain.

12.2.5 Typical Sampling (Meister et al., 2023)

Sample tokens with probability close to the conditional entropy:

$$\mathcal{V}_{\text{typ}} = \{v : |\log p(y_t = v) + H(p(\cdot|y_{<t}))| \leq \tau\} \tag{12.9}$$

Selects "typical" tokens that are neither too surprising nor too obvious. Mathematically grounded in information theory.

12.2.6 Eta (η) Sampling (Hewitt et al., 2022)

Samples tokens whose probability exceeds $\eta \cdot \max_w p(w)$ AND the exponential of the negative entropy:

$$\mathcal{V}_\eta = \{v : p(v) \geq \eta \cdot \max_w p(w)\} \cap \{v : p(v) \geq \alpha \cdot e^{-H(p)}\} \tag{12.10}$$

12.2.7 Repetition Penalties

Reduce probability of already-seen tokens:

- **Repetition penalty:** $z'_v = z_v/\theta$ if v in generated sequence ($\theta > 1$), typically 1.1–1.2.
- **Frequency penalty:** $z'_v = z_v - \alpha \cdot \text{count}(v)$—proportional to occurrence count.
- **Presence penalty:** $z'_v = z_v - \beta$—constant penalty regardless of count.

12.2.8 Mirostat (Basu et al., 2021)

Adaptive temperature control that targets a specific perplexity:

1. **Mirostat v1:** Adjust temperature T_t each step to keep surprise (conditional entropy) within $[2\mu, \mu/2]$ of target τ.
2. **Mirostat v2:** Uses exponential moving averages for more stable adaptation.

No manual temperature tuning needed.

12.3 Constrained and Guided Decoding

12.3.1 Formal Language Constraints

Force model to generate valid JSON, regex patterns, or context-free grammars:

1. Define formal grammar (JSON Schema, regex, CFG).
2. Compile to a finite-state machine (FSM).
3. At each step, compute the set of valid next tokens from the current FSM state.
4. Mask invalid tokens before sampling/argmax.

Libraries:

- **Outlines (Dong et al., 2024):** Compiles grammars to FSM index sets. <10 μs per token overhead.
- **Guidance (Microsoft):** Template-based constrained generation with inline Python control flow.
- **LMQL:** SQL-like query language for LLMs with constraint annotations.
- **llama.cpp grammars:** GBNF (GGML BNF) grammar engine. Compile JSON Schema → GBNF → token-validity masks.
- **SGLang:** Python-native constrained generation via ‘choices‘ and ‘regex‘ parameters.

12.3.2 Constrained Decoding with Outlines

Listing 12.1: JSON-constrained generation

```
import outlines
```

```
model = outlines.models.transformers("mistralai/Mistral-7B-v0.1")
generator = outlines.generate.json(model, MyPydanticModel)
result = generator("Describe a person named Alice")
# result is a valid MyPydanticModel instance
```

12.3.3 Classifier-Free Guidance (Sanchez et al., 2023)

Steer generation toward desired attributes by interpolating conditional and unconditional logits:

$$\tilde{z}_t = z_t^{\text{cond}} + \gamma \cdot (z_t^{\text{cond}} - z_t^{\text{uncond}}) \tag{12.11}$$

where $\gamma > 0$ steers toward attributes specified in the conditioning prompt (z_t^{cond}) and away from the unconditional distribution (z_t^{uncond}).

12.4 Speculative Decoding

12.4.1 Basic Speculative Decoding (Leviathan et al., 2023)

Use a small draft model to propose multiple tokens, then verify with the target model:

1. **Draft:** Small model generates k candidate tokens.
2. **Verify:** Target model computes probabilities for all k prefixes in one forward pass.
3. **Accept/Reject:** For each token position i, accept with probability $\min(1, p_{\text{target}}/p_{\text{draft}})$. If rejected, resample from adjusted target distribution.

Speedup: 2–3× with negligible quality loss (the output distribution is identical to the target model's).

12.4.2 Medusa (Cai et al., 2024)

Instead of a separate draft model, adds multiple **prediction heads** to the target model that predict the next k tokens simultaneously:

$$p^{(i)}(y_{t+i} \mid \mathbf{h}_t) = \text{softmax}(\mathbf{h}_t \mathbf{W}^{(i)}) \tag{12.12}$$

where $\mathbf{W}^{(i)}$ are lightweight linear heads (one per future position). Tree-structured attention verifies multiple candidates in one forward pass.

12.4.3 Eagle (Li et al., 2024)

Feature-level speculative decoding: the draft model predicts future hidden states, not tokens. The target model uses those hidden states to efficiently verify multiple hypotheses.

12.4.4 Self-Speculative Decoding

The target model serves as its own draft model by skipping layers:

1. Compute draft tokens using only first $L/2$ layers of the target model.
2. Verify using all L layers in a single forward pass.

No external draft model needed. Optimized for llama.cpp as "Lookahead Decoding."

12.4.5 Variants and Optimizations

- **Sequoia (Chen et al., 2024):** Dynamic speculation tree with pruning, reducing waste of rejected tokens.
- **Staged speculative decoding:** Multiple draft models of varying size for adaptive speculation.
- **Prompt lookup decoding:** Use n-grams from the prompt as draft tokens (no draft model needed).
- **Recurrent draft models (Rest, He et al., 2024):** An RNN-based draft model with marginal overhead.

12.5 Advanced Sampling and Generation

12.5.1 Locally Typical Sampling (Meister et al., 2022)

Samples tokens whose conditional probability is close to the model's expected information content, ensuring generated text maintains the statistical properties of natural language.

12.5.2 DoLa (Decoding by Contrasting Layers, Chuang et al., 2023)

Contrasts logits from early vs. late transformer layers:

$$\tilde{z}_t = z_t^{(\text{late})} - z_t^{(\text{early})} \tag{12.13}$$

The difference amplifies **factual knowledge** (encoded in later layers) and suppresses **surface-level** linguistic patterns (dominant in early layers). Reduces hallucinations.

12.5.3 Expert Iteration / STaR (Zelikman et al., 2022)

Generate multiple responses, filter for correctness, fine-tune on correct examples, repeat. Self-improving loop: better model → better generations → better fine-tuning data.

12.5.4 Consistency Decoding (CD)

Sample multiple completions and select the most "consistent" one (agreement with other completions) rather than the highest-probability one. Combines with majority voting for improved factuality.

12.5.5 Entropy-Based Decoding

Monitor generation entropy and apply interventions (re-sample, inject context) when entropy spikes, indicating the model is uncertain or going off-track.

12.5.6 Batched Speculative Decoding

Apply speculative decoding across a batch of requests simultaneously, sharing verification overhead across the batch for additional throughput gains in serving scenarios.

Chapter 13

Retrieval-Augmented Generation (RAG)

RAG enhances LLMs by retrieving relevant external knowledge before generating responses, overcoming the model's static knowledge cutoff and hallucination tendencies.

13.1 RAG Fundamentals

13.1.1 Naive RAG Pipeline (Lewis et al., 2020)

1. **Indexing:** Chunk documents, embed them with an encoder, store in a vector database.
2. **Retrieval:** Embed the query, search the vector DB for top-k most similar chunks.
3. **Augmentation:** Insert retrieved chunks into the prompt template.
4. **Generation:** Feed augmented prompt to the LLM.

The standard retrieval approach uses dense embeddings:

$$\text{sim}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \cdot \|\mathbf{d}_i\|} \tag{13.1}$$

where $\mathbf{q} = E_{\text{query}}(x)$ and $\mathbf{d}_i = E_{\text{doc}}(c_i)$ for bi-encoder models.

13.2 Advanced RAG Architectures

13.2.1 Modular RAG (Gao et al., 2023)

Replaces the monolithic naive RAG pipeline with swappable modules:

- **Chunking modules:** Fixed-size, semantic, recursive, or agentic chunking.
- **Embedding modules:** Swappable embedding models (BGE, E5, Instructor, Jina).
- **Retrieval modules:** Dense, sparse (BM25), hybrid, or multi-hop retrievers.
- **Reranking modules:** Cross-encoder rerankers (BGE-reranker, Cohere rerank).
- **Augmentation modules:** Template design, context compression, chunk fusion.
- **Generation modules:** LLM with or without fine-tuning for RAG.

13.2.2 Hybrid RAG (Dense + Sparse)

Combines dense vector search with sparse lexical search (BM25):

$$\text{score}(q, d) = \alpha \cdot \text{sim}_{\text{dense}}(q, d) + (1 - \alpha) \cdot \text{BM25}(q, d) \tag{13.2}$$

Dense retrieval captures semantic similarity; BM25 captures exact keyword matching. Particularly effective for technical/domain-specific queries where keyword precision matters.

13.2.3 Hierarchical RAG (Multi-Level Indexing)

Two-level retrieval:

1. **Coarse level:** Retrieve document summaries (chunked at the section/paragraph level).
2. **Fine level:** Within selected summaries, retrieve detailed chunks for context.

Improves retrieval precision by first identifying relevant documents, then drilling into specific sections. Reduces noise from superficially similar but irrelevant chunks.

13.2.4 Self-RAG (Asai et al., 2023)

The LLM actively decides **when** to retrieve and **critiques** its own generations:

1. For each segment, predict a special `<Retrieve>` token if external knowledge is needed.
2. Retrieve and generate with retrieved context.
3. Predict reflection tokens (`<IsRel>`, `<IsSup>`, `<IsUse>`) to self-assess relevance, support, and usefulness.
4. If unsatisfactory, re-generate with different retrievals.

13.2.5 Corrective RAG (CRAG, Yan et al., 2024)

Retrieval evaluator assesses quality before generation:

1. Retrieve documents.
2. **Evaluator** scores each retrieved document's relevance.
3. **High relevance:** Use directly for generation.
4. **Low relevance:** Trigger web search for additional knowledge.
5. **Mixed relevance:** Combine retrieved + web-searched knowledge.

The **decompose-then-recompose** algorithm breaks complex queries into sub-queries, retrieves for each, and synthesizes.

13.3 ColBERT and Late Interaction Retrieval

13.3.1 ColBERT v1/v2 (Khattab & Zaharia, 2020)

Unlike bi-encoders (which produce single-vector representations), ColBERT computes token-level interactions:

$$\text{score}(\mathbf{q}, \mathbf{d}) = \sum_{i=1}^{|\mathbf{q}|} \max_{j=1}^{|\mathbf{d}|} \mathbf{q}_i \cdot \mathbf{d}_j^\top \tag{13.3}$$

Each query token matches with the most similar document token (**MaxSim**), and the per-token scores are summed. This captures fine-grained token-level relevance while remaining scalable via late interaction (interaction happens after encoding).

ColBERT v2: Adds a residual compression mechanism that reduces the index size by discarding low-information tokens while preserving retrieval quality. Also introduces centroid-based clustering for approximate search.

13.3.2 SPLADE (Sparse Lexical and Expansion)

Learns sparse, interpretable query/document representations:

$$\mathbf{w}_q = \text{MLM}_{\text{head}}(\text{BERT}(q)), \quad \text{score} = \sum_{t \in V} \max(w_{q,t}, w_{d,t}) - w_{q,t} \cdot w_{d,t} \tag{13.4}$$

Combines the semantic understanding of dense models with the interpretability of sparse (bag-of-words) models. Vocabulary expansion discovers related terms.

13.3.3 PLAID (Efficient ColBERT)

Engine for efficient ColBERT retrieval using centroid-based candidate generation, centroid interaction, and IVF-based filtering. Achieves <100ms latency over millions of documents.

13.4 Retrieval Optimizations

13.4.1 Re-ranking with Cross-Encoders

For each of the top-k retrieved candidates, the cross-encoder computes a joint encoding of the query and document:

$$s_i = \text{CrossEncoder}([q; d_i]) \tag{13.5}$$

More accurate than bi-encoder scoring but too slow for full-corpus search. Applied as a second stage on ∼100–1000 candidates.

13.4.2 Query Transformations

- **Query expansion (HyDE):** Generate a hypothetical answer, embed it, and retrieve documents similar to the answer (not the question). Based on the hypothesis that answers are more similar to relevant documents than questions are.
- **Multi-query retrieval:** Generate k diverse rephrasings of the query, retrieve for each, deduplicate results.
- **Step-back prompting:** Ask a broader, more abstract question to retrieve background context, then answer the specific question.
- **Query2doc:** Generate a pseudo-document from the query using the LLM, concatenate it with the query for retrieval.

13.4.3 Context Compression (LLMLingua)

Compress retrieved contexts before feeding to the LLM:

1. Identify and remove low-information tokens (using perplexity-based metrics).
2. Keep only tokens essential for answering the query.
3. Reduces prompt length and computational cost while preserving answer quality.

13.4.4 Fusion / Reciprocal Rank Fusion (RRF)

Combine results from multiple retrieval methods:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \tag{13.6}$$

where $\mathcal{R}$ is the set of retrieval systems and k is a constant (typically 60). No tuning needed—inherently balances different retrievers.

13.5 Embedding Models for RAG

13.5.1 Bi-Encoder Architectures

Most effective embedding models for retrieval:

- **BGE (BAAI General Embedding):** BERT-based, trained with RetroMAE pre-training + contrastive learning. Strong zero-shot and fine-tuned performance.
- **E5 (EmbEddings from bidirEctional Encoder representAtions):** Two-stage: weakly-supervised contrastive pre-training on CCPairs, then supervised fine-tuning.
- **Instructor (Su et al., 2022):** Instruction-tuned embeddings—the embedding depends on a task instruction. "Represent the Wikipedia article for retrieval" vs. "Represent the question for retrieving supporting documents."
- **GTE (General Text Embeddings):** Unified model handling classification, clustering, pair classification, reranking, and retrieval.
- **Jina Embeddings v2:** 8K context window, multilingual, optimized for retrieval.
- **Voyage AI:** Optimized for specific domains (code, legal, finance).

13.5.2 Embedding Tuning and Matryoshka Embeddings

- **Instruction-tuned embeddings:** Generate different embeddings for the same text by conditioning on task-specific instructions.
- **Matryoshka Representation Learning:** Learn embeddings where the first $d' < d$ dimensions also form a valid (if slightly less accurate) embedding. Enables adaptive dimensionality for storage savings.
- **Angle-optimized embeddings:** Optimize the cosine angle rather than MSE during training (better for cosine-similarity retrieval).

13.6 Multi-Hop and Agentic RAG

13.6.1 Multi-Hop Retrieval

For complex queries requiring synthesizing information from multiple documents:

1. Retrieve initial documents.
2. Generate intermediate reasoning (or sub-questions).
3. Retrieve additional documents based on intermediate findings.
4. Repeat until sufficient information is gathered.

Implemented via **IRCoT** (Interleaving Retrieval with Chain-of-Thought): generates a reasoning step, then retrieves based on that step, then generates the next step.

13.6.2 Agentic RAG (ARAG)

An LLM agent orchestrates the retrieval process:

- **Tool use:** Agent has access to multiple retrieval tools (vector DB, web search, SQL query, API calls).
- **Plan-then-execute:** Agent formulates a retrieval plan, executes it, evaluates results, and iterates if needed.
- **Self-reflection:** Agent critiques its own retrieval quality and adjusts strategy.

13.6.3 Graph RAG (Microsoft, 2024)

Uses knowledge graphs constructed from source documents:

1. Extract entities and relationships from documents to build a knowledge graph.
2. Summarize communities (clusters) in the graph.
3. For a query, retrieve relevant community summaries.
4. Use the summaries as context for LLM generation.

Particularly effective for global sensemaking ("What are the main themes in this dataset?").

13.7 Multimodal RAG

Extends RAG to images, tables, and other modalities:

- **MuRAG:** Retrieves text AND images from a multimodal knowledge base using CLIP-like embeddings.
- **Table RAG:** Retrieves structured data (tables, CSV) using table-aware embeddings.
- **Code RAG:** Retrieves relevant code snippets from a codebase using code-specific embeddings (e.g., CodeBERT-based).

13.8 RAG Evaluation Metrics

- **Faithfulness:** Does the generated answer faithfully reflect retrieved context (not hallucinate)?
- **Answer relevance:** Does the answer address the query?
- **Context relevance:** Are the retrieved chunks relevant to the query?
- **Context recall:** Does retrieval find all necessary information?
- **RAGAS (RAG Assessment):** Automated evaluation framework using LLM-as-judge for the above metrics.

13.9 Caching Strategies for RAG

- **Semantic cache:** Cache query-answer pairs based on embedding similarity rather than exact string match.
- **Prefix caching:** Share KV-cache for repeated prompt prefixes (common in RAG where template prefixes are identical).
- **Chunk caching:** Cache frequently retrieved chunks' embeddings and reranking scores.

Chapter 14

Hooking, Model Interpretability, and Scheduling

This chapter covers techniques for intercepting and analyzing model internals, plus scheduling strategies for efficient multi-request serving.

14.1 Hooking: Intercepting Model Internals

14.1.1 PyTorch Forward Hooks

The foundational mechanism for intercepting activations:

Listing 14.1: PyTorch forward hook

```
def hook_fn(module, input, output):
    # Intercept the output of a module
    print(f"Output shape: {output.shape}")
    # Can store, modify, or analyze activations

handle = model.layers[10].attention.register_forward_hook(hook_fn)
output = model(input)
handle.remove()  # Clean up
```

Hook types: `register_forward_hook` (output), `register_forward_pre_hook` (input), `register_full_backward_hook` (gradients), `register_state_dict_pre_hook` (state modifications).

14.1.2 TransformerLens (Nanda & Bloom, 2022)

A library specifically designed for mechanistic interpretability of transformer models:

- **Cached activations:** Automatically caches residual stream, attention patterns, MLP outputs, and neuron activations at every layer.
- **Activation patching:** Intervene on specific activations (e.g., set a neuron's output to zero at a specific position) and measure the effect on the output.
- **Logit attribution:** Trace which components (attention heads, MLP neurons) contributed to a specific token prediction.
- **Ablation experiments:** Zero out or scramble specific attention heads or MLP layers to understand their function.

- **Path patching:** Trace the causal pathway between specific components across layers.

Listing 14.2: TransformerLens basic usage

```
from transformer_lens import HookedTransformer

model = HookedTransformer.from_pretrained("gpt2-small")
logits, cache = model.run_with_cache("Hello, world!")
# cache["blocks.5.attn.hook_pattern"] contains attention patterns
# cache["blocks.3.mlp.hook_post"] contains MLP activations
# Activation patching: modify a head's output and see effect
```

14.1.3 NNsight (Jaden, 2024)

A PyTorch-based library for remote execution and intervention:

- **Tracing context:** Define interventions in a "tracing context" that specifies which layers to intercept and what operations to perform.
- **Remote execution:** Run on remote models (loaded on GPU servers) while writing intervention code locally.
- **Cross-prompt interventions:** Patch activations from one prompt into another during forward pass.

14.1.4 Steering Vectors / Activation Engineering

Modify model behavior by adding bias vectors to activations:

1. Collect activations for desired behavior (e.g., "truthful responses") and undesired behavior (e.g., "hallucinated responses").
2. Compute the difference: $\mathbf{v}_{\text{steer}} = \bar{\mathbf{a}}_{\text{desired}} - \bar{\mathbf{a}}_{\text{undesired}}$.
3. At inference, add scaled steering vector: $\mathbf{a}' = \mathbf{a} + \alpha \cdot \mathbf{v}_{\text{steer}}$.

Applications: truthfulness enhancement, refusal vector removal, emotional steering, and style transfer.

14.1.5 Representation Engineering (RepE, Zou et al., 2023)

A framework for reading and controlling high-level concepts in LLM representations:

- **Reading:** Linear probe on activations to detect concepts (honesty, power-seeking, emotion).
- **Control:** Apply LoRA-like low-rank perturbations to activations to steer behavior.

14.1.6 Refusal Vector Ablation (Arditi et al., 2024)

Identify and remove the "refusal direction" in model activation space:

1. Collect activations from the model when it refuses a harmful request.
2. Find the principal direction of refusal activations via PCA or mean differencing.
3. At inference, project activations to remove this direction: $\mathbf{a}' = \mathbf{a} - (\mathbf{a} \cdot \mathbf{v}_{\text{refuse}})\mathbf{v}_{\text{refuse}}$.

This can remove safety guardrails without fine-tuning, raising important ethical considerations.

14.1.7 Logit Lens and Tuned Lens

- **Logit Lens:** Apply the unembedding matrix to intermediate layer representations to "read" what the model would predict at each layer, revealing how predictions form progressively.
- **Tuned Lens (Belrose et al., 2023):** A trained affine transformation that maps each layer's representation to a better approximation of the final output. Reveals **latent predictions** at each layer with higher accuracy than the logit lens.

14.1.8 SAE (Sparse Autoencoder) for Feature Decomposition

Train sparse autoencoders on model activations to decompose them into interpretable features:

$$\mathbf{a} = \text{Decoder}(\text{Encoder}(\mathbf{a})), \quad \text{Encoder is sparse} \tag{14.1}$$

The sparse latent codes reveal which high-level concepts activate at specific positions. Used by Anthropic and OpenAI for model interpretability research.

14.1.9 Patching and Causal Tracing

- **Activation patching:** Replace the activation at location (layer, position) with a corrupted version and measure the effect.
- **Causal tracing (Meng et al., 2022):** Systematically patch across layers/positions to identify the **causal pathway** of factual recall. Found that MLP layers at the last subject token are crucial for fact extraction.
- **Knockout analysis:** Remove specific circuit components and measure task degradation.

14.2 Scheduling and Batching Strategies

14.2.1 Static Batching

Wait for all requests in a batch to finish before starting the next batch:

- **Padding waste:** Shorter sequences must be padded to batch max length.
- **Tail latency:** One long request holds up all shorter ones.
- **GPU utilization:** 30–50% typical (poor due to idle time waiting for slowest request).

14.2.2 Continuous Batching (Iteration-Level Scheduling)

Add/remove requests on every forward pass (per-token granularity):

1. **Preemption:** Running requests can be temporarily paused (KV-cache saved) if higher-priority requests arrive.
2. **Swapping:** KV-cache blocks can be swapped to CPU memory and back when VRAM constrained.
3. **Recomputation:** If CPU memory is also tight, recompute KV-cache from scratch when needed.

Implementation in vLLM **scheduler**:

Listing 14.3: vLLM scheduling logic (conceptual)

```
class Scheduler:
    def schedule(self) -> SchedulerOutputs:
        # 1. Remove completed sequences
        # 2. Allocate KV-cache blocks for new requests
        # 3. If OOM, preempt lowest-priority sequences
        # 4. Select running + new sequences that fit
        # 5. Return the batch for this iteration
        ...
```

14.2.3 Chunked Prefill Scheduling

Long prompts are split into chunks and interleaved with decode steps:

- Prevents long prefills from starving decode (improves TTFT).
- Each chunk processes C tokens of a pending prompt.
- One decode step interleaved per chunk.
- **Splitwise (Patel et al., 2024):** Separates prefill and decode onto different GPUs entirely, dedicating specialized hardware to each phase.

14.2.4 Priority-Based Scheduling

- **FCFS (First-Come First-Served):** Simple. High latency variance.
- **SJF (Shortest Job First):** Prioritize short requests. Minimizes average latency. Risk: long requests may starve.
- **Multi-level feedback queue:** Multiple queues at different priority levels. Requests start at high priority, progressively demoted.
- **Fair scheduling:** Proportional allocation. vLLM uses water-filling: allocate tokens proportional to arrival rates.
- **Deadline-aware:** Prioritize requests approaching their SLO (Service Level Objective) deadline.

14.2.5 Request Scheduling for LoRA Serving

When serving multiple LoRA adapters concurrently:

- **LoRA-aware batching:** Group requests using the same LoRA adapter into shared micro-batches to amortize the cost of loading adapter weights.
- **Adapter caching:** Keep frequently-used adapters in GPU memory, evict least-used adapters.
- **S-LoRA (Sheng et al., 2023):** Scalable serving of thousands of LoRA adapters via unified memory management for adapters.

14.2.6 Power Management and Thermal Throttling

For continuous local deployment:

- **Dynamic frequency scaling:** Reduce GPU clock when idle or under light load.
- **Temperature-aware scheduling:** Distribute load across GPUs to avoid thermal hotspots.

- **NVML-based monitoring:** Track GPU utilization, temperature, and memory in real-time to adjust scheduling decisions.

14.2.7 Speculative Scheduling

Predict request characteristics before completion:

- **Length prediction:** Use a lightweight classifier to predict total output length from the prompt.
- **Adaptive prefill sizing:** Adjust chunk size based on predicted output length.
- **Predictive KV-cache allocation:** Allocate more cache blocks for requests likely to generate long outputs.

Chapter 15

Domain Expert Small LLMs

Small language models ($\leq$7B parameters) are the backbone of local deployment. This chapter covers model families, domain specialization strategies, training recipes, and practical deployment considerations.

15.1 Major Small LLM Families

15.1.1 LLaMA Family (Meta)

- **LLaMA-2 7B:** Foundation for most local deployment. RMSNorm, SwiGLU, RoPE, GQA (in 70B). Strong open-weight baseline.
- **LLaMA-3 8B:** Improved tokenizer (128K vocab), higher RoPE frequency, GQA ($g = 8$). Outperforms LLaMA-2 13B.
- **LLaMA-3.1 8B:** 128K context via RoPE interpolation. Multilingual. Strong instruction-following capabilities.

15.1.2 Mistral (Mistral AI)

- **Mistral 7B:** Sliding window attention (4096), GQA. Competitive with LLaMA-2 13B.
- **Mistral Nemo 12B:** Tekken tokenizer, 128K context. Joint NVIDIA/Mistral training on DGX Cloud.
- **Codestral 22B:** Code-specialized variant. Fill-in-the-middle training for code completion.

15.1.3 Qwen Series (Alibaba)

- **Qwen2-7B:** GQA, SwiGLU, tied embeddings. Strong multilingual performance.
- **Qwen2.5-7B:** 32K context, improved pre-training data recipe. Matches LLaMA-3 8B.
- **Qwen2.5-Coder 7B:** Code-specialized. Fill-in-the-middle and repository-level code understanding.
- **Qwen2.5-Math 7B:** Math-specialized with step-by-step reasoning.

15.1.4 Gemma (Google)

- **Gemma 2B / 7B:** GeGLU activations (GELU gating instead of SiLU), 256K vocab. Strong for size.
- **Gemma-2 9B / 27B:** Interleaving local:global attention. Knowledge distillation from larger teachers.
- **CodeGemma / RecurrentGemma:** Domain-specific and SSM-based variants.

15.1.5 Phi Series (Microsoft)

- **Phi-1 / Phi-1.5:** "Textbooks are all you need"—trained on synthetic textbook-quality data. 1.3B surpassing much larger models on code tasks.
- **Phi-2:** 2.7B. Synthetic training data + data pruning. High-quality small model for reasoning.
- **Phi-3 Mini (3.8B):** 4K/128K context variants. Knowledge distillation approach.
- **Phi-3.5 Mini:** MoE variant with 3.8B active/6.6B total. Strong multi-lingual tasks.

15.1.6 DeepSeek

- **DeepSeek-Coder 1.3B/6.7B:** Trained from scratch on 2T code tokens from GitHub + StackOverflow. MLM-style fill-in-the-middle.
- **DeepSeek-V2-Lite:** 16B total with 2.4B active (MoE). MLA (Multi-head Latent Attention) for KV-cache compression.

15.1.7 Additional Notable Small LLMs

- **SmolLM (HuggingFace):** 135M, 360M, 1.7B. Trained on SmolTalk/Cosmopedia datasets. High-quality tiny models.
- **TinyLlama:** 1.1B. Trained on 3T tokens. Open reproduction of LLaMA at small scale.
- **StableLM (Stability AI):** 3B, Zephyr series (DPO-tuned). Compact open-source options.
-
- **OLMo (AI2):** 1B, 7B. Fully open (data, code, weights, logs). Transparency-focused.
- **Falcon-Mamba 7B:** SSM-based (pure Mamba architecture, no attention). Alternative architecture path.
- **H2O-Danube:** 1.8B. Trained on 1T tokens by H2O.ai for enterprise deployment.

15.2 Domain Specialization Strategies for Small LLMs

15.2.1 Continued Pre-Training (CPT)

Continue training a pre-trained model on domain-specific data:

1. Start from general pre-trained checkpoint.
2. Mix domain data with general data to prevent catastrophic forgetting.
3. Typical ratio: 80% domain / 20% general replay.

4. Learning rate: ∼10% of original pre-training LR.
5. Steps: 10K–100K depending on domain data volume.

Domain examples:

- **Legal:** Fine-tune on case law, statutes, legal opinions. SaulLM, LexiLaw.
- **Medical:** Fine-tune on PubMed, clinical notes, medical textbooks. PMC-LLaMA, Meditron.
- **Finance:** Fine-tune on SEC filings, financial reports, FinPile. FinGPT, BloombergGPT (proprietary).
- **Code:** Fine-tune on GitHub, StackOverflow, code documentation. CodeLlama, StarCoder.
- **Science:** Fine-tune on arXiv, textbooks, research papers. Galactica, SciGLM.

15.2.2 Instruction Fine-Tuning for Domain Tasks

Curate domain-specific instruction data:

1. **Self-Instruct:** Use a strong LLM to generate domain-specific (instruction, response) pairs from seed tasks.
2. **Evol-Instruct:** Iteratively increase complexity of instructions through multi-step refinement.
3. **Expert annotation:** Have domain experts create high-quality demonstration pairs.
4. **Synthetic data generation:** Generate training data from textbooks, documentation, and Q&A forums.

15.2.3 Retrieval-Augmented Domain Models

Combine a small LLM with a domain-specific knowledge base:

- **Domain-specific vector DB:** Store domain documents as embeddings. Retrieve relevant passages for each query.
- **Knowledge graph integration:** Structured domain knowledge (entity relationships, taxonomies) converted to retrievable text chunks.
- **API tool integration:** Give the model access to domain-specific tools (SQL queries, calculators, code interpreters).

15.2.4 Ensemble and Specialization Routing

When multiple domain-specific small models exist:

- **Prompt-based routing:** Analyze query keywords to route to the appropriate specialized model.
- **Embedding-based routing:** Embed the query and find the nearest domain centroid.
- **LLM-as-router:** A tiny classifier model that predicts the best domain model for a query.

15.3 Training Recipes for Small LLMs

15.3.1 Data Quality and Curation

The quality of training data disproportionately affects small models:

- **Deduplication:** MinHash LSH for near-duplicate detection within and across documents.
- **Quality filtering:** Perplexity-based filtering (remove too-repetitive or too-random text), classifier-based filtering (keep educational/helpful content).
- **Toxicity/PII filtering:** Remove NSFW content and personally identifiable information.
- **Data mixing ratios:** Balance between web text, books, code, academic papers, and conversational data.
- **Phi-style synthetic data:** Generate textbook-quality data from larger models for pre-training.

15.3.2 Training Efficiency Techniques

- **Flash Attention 2:** Enables longer context training on smaller GPUs.
- **ZeRO (DeepSpeed):** ZeRO-2 for optimizer state partitioning; ZeRO-3 for full parameter partitioning.
- **FSDP (PyTorch):** Fully Sharded Data Parallel for distributed training.
- **Gradient checkpointing:** Trade compute for memory—recompute activations during backward pass.
- **Mixed precision:** BF16 training with FP32 master weights.
- **Packing:** Pack multiple short documents into single sequences to maximize GPU utilization.

15.3.3 LR Schedules and Hyperparameters

For small LLM training:

- **Learning rate:** Typically 3×10^{-4} for 1B-class models (with warmup).
- **Warmup:** 2000 steps linear warmup, then cosine decay to 10^{-5}.
- **Batch size:** 4M tokens global batch size, with gradient accumulation for smaller GPUs.
- **Weight decay:** 0.1 applied only to non-bias/non-norm parameters.
- **Gradient clipping:** 1.0 max norm for stability.

15.3.4 Chinchilla-Optimal vs Over-Training

- **Chinchilla scaling law:** Optimal tokens-to-parameters ratio is ∼20:1 (e.g., 1B params → 20B tokens).
- **Over-training (modern):** Training small models on many more tokens (e.g., TinyLlama 1.1B on 3T tokens = 2700:1 ratio). Large gains in performance per parameter from over-training.
- **Key insight:** For local deployment, compute spent on training is amortized; inference compute is the bottleneck. Over-training small models yields better "inference efficiency."

15.4 Locally Deployable Domain Expert Architecture

15.4.1 Quantized Domain Model Stack

A typical local domain expert deployment:

1. **Base model:** Qwen-2.5 7B or LLaMA-3 8B in Q4_K_M or AWQ INT4 (∼4.5 GB).
2. **Domain adapter:** LoRA rank 32 trained on domain-specific instruction data (∼20 MB).
3. **Vector database:** ChromaDB or LanceDB with ∼10K domain document chunks (∼200 MB).
4. **Total:** ∼5 GB VRAM/RAM for a functional domain expert.

15.4.2 Edge Deployment (Raspberry Pi / Mobile)

For extreme resource constraints:

- **Model:** SmolLM 360M or Qwen2-0.5B in Q4_0 (∼300 MB).
- **Engine:** llama.cpp with ARM NEON or Vulkan backend.
- **KV-cache:** 4-bit quantized, with sliding window for fixed memory.
- **Throughput:** 5–15 tokens/sec on Raspberry Pi 5.

15.4.3 Local RAG + LLM Stack

Listing 15.1: Local RAG with ChromaDB + llama.cpp

```
import chromadb
from llama_cpp import Llama

# Embedding model for retrieval
embed_model = Llama.from_pretrained(
    repo_id="nomic-ai/nomic-embed-text-v1.5-GGUF",
    filename="*Q8_0.gguf", embedding=True
)

# Vector database
client = chromadb.PersistentClient(path="./domain_db")
collection = client.get_or_create_collection("docs")
# Index documents...
collection.add(documents=chunks, embeddings=embed_model.embed(
    chunks))

# Query
results = collection.query(query_embeddings=embed_model.embed([
    query]))
context = "\n".join(results["documents"][0])

# LLaMA for generation
llm = Llama(model_path="model-Q4_K_M.gguf", n_ctx=4096)
response = llm(f"Context:\n{context}\n\nQuestion: {query}\nAnswer:
    ")
```

15.5 Future Directions for Small LLMs

- **MatMul-free architectures:** Replace all matrix multiplications with ternary accumulations (BitNet b1.58). Extremely efficient on edge hardware.
- **1-bit LLMs:** Quantize to 1–1.5 bits with negligible quality loss through new quantization paradigms.
- **Speculative decoding from small to smaller:** Use a 100M speculative model to accelerate a 1B model 2–3×.
- **Online learning:** Continuously update domain knowledge via on-device fine-tuning without full re-training.
- **Neuro-symbolic fusion:** Combine small LLMs with symbolic reasoners for hybrid capabilities within tight resource budgets.
- **Federated domain adaptation:** Multiple edge devices collaboratively fine-tune domain models while preserving privacy.

15.6 Small LLM Selection Guide

Table 15.1: Model selection by deployment scenario

Scenario	**Recommended Model**	**Quantization**
8GB GPU (RTX 3070)	LLaMA-3 8B / Mistral 7B	Q4_K_M (5 GB)
4GB GPU (GTX 1650)	Phi-3 Mini 3.8B / Qwen-2.5 3B	Q4_K_M (2.5 GB)
16GB RAM (CPU-only)	LLaMA-3 8B	Q4_K_M
8GB RAM (CPU-only)	Phi-3 Mini / Gemma-2 2B	Q4_0 (1.5 GB)
Apple M1 8GB	Mistral 7B / LLaMA-3 8B	Q4_K_M (Metal)
Raspberry Pi 5	SmolLM 360M / Qwen 0.5B	Q4_0 (300 MB)
Code generation	DeepSeek-Coder 6.7B	Q4_K_M
Multilingual	Qwen-2.5 7B / Aya-23 8B	Q4_K_M

Afterword

This book has covered the complete pipeline for running LLMs locally—from the foundational architecture decisions (RMSNorm, SwiGLU, RoPE, GQA) through compression (quantization, pruning, distillation), efficient fine-tuning (PEFT), alignment (DPO, SimPO), domain specialization, inference optimization, retrieval augmentation, and model analysis.

The key takeaway: local LLM deployment is not about a single silver bullet, but about **composability**. Each technique serves a specific purpose, and they multiply in effectiveness when combined. A quantized model with GQA, served by an engine with PagedAttention and continuous batching, retrieved over a domain-specific vector database, with speculative decoding accelerating generation—each piece is individually manageable, but together they enable capabilities that would be impossible with any single technique alone.

The field moves rapidly, but the mathematical and algorithmic foundations covered here will remain relevant as architectures evolve. Understanding *why* SwiGLU works better than ReLU, *how* RoPE encodes relative position, and *what* the online softmax algorithm achieves will serve you regardless of the next architecture paradigm.

Happy building.

— *Comprehensive Guide to Running LLMs Locally, May 2026*